

第7章 微调

微调是通过进一步训练整个模型或部分模型，使其适应特定任务的过程。第5章和第6章讨论了基于提示的方法，这些方法通过给模型提供指令、上下文和工具来使其适应任务。而微调则是通过调整模型的权重来适应任务。

微调可以增强模型的多个方面。它可以改进模型在特定领域的能力，如编程或医疗问答，也可以增强模型的安全性。但最常见的用途是改进模型遵循指令的能力，特别是确保它遵循特定的输出风格和格式。

虽然微调可以帮助创建更符合您需求的定制模型，但它也需要更多的前期投入。我经常被问到的一个问题是：何时应该微调，何时应该做RAG？在概述微调之后，本章将讨论微调的理由和不微调的理由，以及在微调和替代方法之间选择的简单框架。

与基于提示的方法相比，微调需要更大的内存占用。以当今基础模型的规模，单纯的微调通常要比单个GPU可用内存更多的内存。这使得微调昂贵且具有挑战性。正如本章所讨论的，减少内存需求是许多微调技术的主要动机。本章专门用一节来概述影响模型内存占用的因素，这对理解这些技术很重要。

在微调领域中占主导地位的一种内存高效方法是PEFT(参数高效微调)。本章探讨PEFT及其与传统微调的不同之处，并概述了其不断发展的技术。我将特别关注一类引人注目的技术：基于适配器的技术。

对于基于提示的方法，了解ML模型在底层的运作方式是推荐的，但不是绝对必要的。然而，微调将您带入模型训练领域，这里需要ML知识。ML基础知识超出了本书的范围。如果您想快速复习，本书的GitHub仓库中有指向有用资源的链接。在本章中，我将介绍与讨论直接相关的几个核心概念。

这是我写作中技术上最具挑战性的一章，不是因为概念的复杂性，而是因为这些概念涵盖的广泛范围。我猜想读起来可能也具有技术挑战性。如果在任何时候，您觉得自己深入到与您的工作无关的细节中，请随时跳过。

有很多内容要讨论。让我们开始吧！

微调概述

进行微调时，您从一个具有一些(但不是全部)所需能力的基础模型开始。微调的目标是让这个模型在您的特定任务上表现得足够好。

微调是进行迁移学习的一种方式，这个概念最初由Bozinovski和Fulgosi在1976年提出。迁移学习关注如何将从一个任务中获得的知识转移，以加速学习新的相关任务。这在概念上类似于人类转移技能的方式：例如，会弹钢琴可以使学习另一种乐器变得更容易。

迁移学习的早期大规模成功是谷歌的多语言翻译系统(Johnson等, 2016)。该模型将其葡萄牙语-英语和英语-西班牙语翻译的知识转移到直接翻译葡萄牙语到西班牙语，尽管训练数据中没有葡萄牙语-西班牙语的例子。

自深度学习早期以来，迁移学习为数据有限或获取成本高的任务提供了解决方案。通过在数据丰富的任务上训练基础模型，然后将这些知识转移到目标任务上。

对于LLM，从预训练文本补全(一个数据丰富的任务)中获得的知识被转移到更专业的任务上，如法律问答或文本到SQL转换，这些任务通常可用数据较少。这种迁移学习能力使基础模型特别有价值。

迁移学习提高了样本效率，使模型能够用更少的例子学习相同的行为。样本高效的模型可以从更少的样本中有效学习。例如，从头开始训练法律问答模型可能需要数百万个例子，而微调一个好的基础模型可能只需要几百个。

理想情况下，模型需要学习的大部分内容已经存在于基础模型中，微调只是对模型行为进行细化。OpenAI的InstructGPT论文(2022)建议将微调视为释放模型已有但用户仅通过提示难以访问的能力。

注意

微调不是进行迁移学习的唯一方式。另一种方法是基于特征的迁移。在这种方法中，训练一个模型从数据中提取特征，通常作为嵌入向量，然后由另一个模型使用。我在第2章简要提到了基于特征的迁移，当讨论如何通过添加分类器头部来重用基础模型的一部分进行分类任务时。

基于特征的迁移在计算机视觉中非常常见。例如，在2010年代后半期，许多人使用在ImageNet数据集上训练的模型从图像中提取特征，并在其他计算机视觉任务(如物体检测或图像分割)中使用这些特征。

微调是模型训练过程的一部分，是模型预训练的延伸。由于预训练后发生的任何训练都是微调，微调可以采取多种不同形式。第2章已经讨论了两种微调类型：监督微调和偏好微调。让我们快速回顾这些方法以及作为应用开发者如何利用它们。

回顾一下，模型的训练过程始于预训练，通常采用自监督方式。自监督允许模型从大量无标签数据中学习。对于语言模型，自监督数据通常只是不需要注释的文本序列。

在使用昂贵的特定任务数据微调这个预训练模型之前，你可以使用廉价的任务相关数据通过自监督对其进行微调。例如，要微调一个法律问答模型，在用昂贵的标注(问题，答案)数据微调之前，

可以先在原始法律文档上进行微调。同样，要微调一个用越南语进行图书摘要的模型，你可以先在大量越南语文本上微调它。自监督微调也称为持续预训练。

如第1章所述，语言模型可以是自回归或掩码式的。自回归模型使用前面的标记作为上下文来预测序列中的下一个标记。掩码模型使用前后的标记来填充空白。同样，通过监督微调，你也可以微调模型来预测下一个标记或填补空白。后者也被称为填充微调，对于文本编辑和代码调试等任务特别有用。即使模型是自回归预训练的，你也可以将其微调为填充任务。

模型在自监督学习期间可以学习的大量数据使其具备丰富的世界知识，但用户可能很难为自己的任务提取这些知识，或者模型的行为可能与人类偏好不一致。监督微调使用高质量的标注数据来调整模型，使其与人类使用和偏好保持一致。

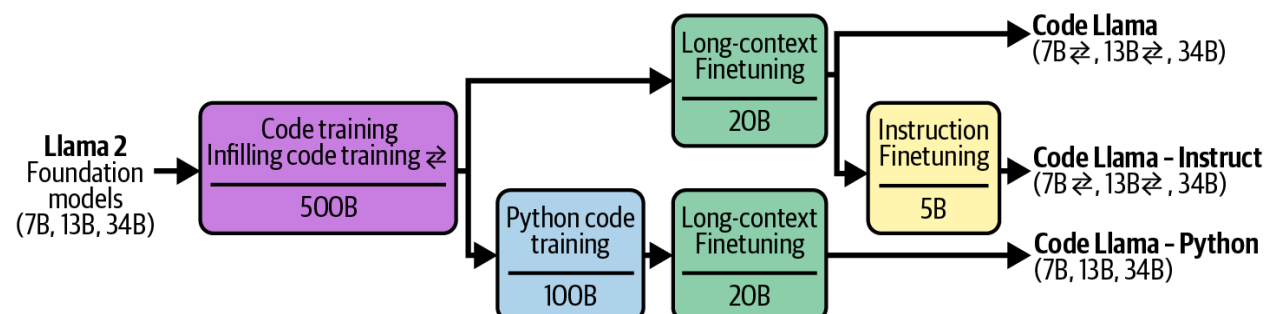
在监督微调过程中，模型使用(输入，输出)对进行训练：输入可以是指令，输出可以是响应。响应可以是开放式的，如图书摘要任务，也可以是封闭式的，如分类任务。创建高质量的指令数据可能具有挑战性且昂贵，特别是对于需要事实一致性、领域专业知识或政治正确性的指令。第8章讨论了如何获取指令数据。

模型还可以通过强化学习进行微调，以生成能够最大化人类偏好的响应。偏好微调需要比较数据，通常遵循(指令，胜出响应，失败响应)的格式。

可以微调模型以扩展其上下文长度。长上下文微调通常需要修改模型的架构，如调整位置嵌入。更长的序列意味着标记可能出现的位置更多，位置嵌入应该能够处理这些情况。与其他微调技术相比，长上下文微调更难实现。所得到的模型在较短序列上的性能可能也会下降。

图7-1显示了不同Code Llama模型的制作过程(Rozière等, 2024)，从基础模型Llama 2开始，使用不同的微调技术。使用长上下文微调，他们能够将模型的最大上下文长度从4,096个标记增加到16,384个标记，以适应更长的代码文件。在图中，指令微调指的是监督微调。

模型开发者和应用开发者都可以进行微调。模型开发者通常在发布模型前使用不同的微调技术对模型进行后训练。模型开发者还可能发布经过不同程度微调的不同模型版本，让应用开发者可以选择最适合他们的版本。



作为应用开发者，你可能会微调预训练模型，但更可能的是，你会微调已经经过后训练的模型。模型越精细，其知识与你的任务越相关，你需要做的适应工作就越少。

何时进行微调

在深入了解不同的微调技术之前，有必要考虑微调是否适合你。与基于提示的方法相比，微调需要更多的资源，不仅在数据和硬件方面，还有ML人才方面。因此，微调通常在对基于提示的方法进行广泛实验后才会尝试。然而，微调和提示不是互斥的。现实世界的问题通常需要两种方法。

微调的理由

微调的主要原因是提高模型的质量，包括一般能力和特定任务能力。微调通常用于改进模型生成遵循特定结构的输出的能力，如JSON或YAML格式。

一个在广泛基准测试中表现良好的通用模型可能在你的特定任务上表现不佳。如果你想使用的模型在你的任务上训练不足，用你的数据对其进行微调会特别有用。

例如，一个现成的模型可能擅长将文本转换为标准SQL方言，但可能在不太常见的SQL方言上失效。在这种情况下，用含有这种SQL方言的数据微调这个模型会有所帮助。同样，如果模型对标准SQL的常见查询表现良好，但对客户特定查询经常失败，微调模型处理客户特定查询可能会有所帮助。

微调的一个特别有趣的用例是偏见缓解。如果基础模型从其训练数据中延续了某些偏见，在微调过程中暴露于精心策划的数据可以抵消这些偏见(Wang和Russakovsky, 2023)。例如，如果模型始终给CEO分配男性名字，在有許多女性CEO的数据集上微调它可以减轻这种偏见。Garimella等人(2022)发现，在女性撰写的文本上微调类BERT语言模型可以减少这些模型的性别偏见，而在非洲作者的文本上微调可以减少种族偏见。

你可以微调一个大模型使其变得更好，但微调较小的模型更为常见。较小的模型需要更少的内存，因此更容易微调。它们在生产中使用也更便宜、更快。

一种常见的方法是微调一个小模型来模仿较大模型的行为，使用由这个大模型生成的数据。因为这种方法将较大模型的知识提炼到较小的模型中，它被称为蒸馏。这将在第8章与其他数据合成技术一起讨论。

在特定任务上微调的小模型可能会超过现成的大得多的模型。例如，Grammarly发现他们微调的Flan-T5模型(Chung等, 2022)在各种写作助手任务上的表现优于专门用于文本编辑的GPT-3变体，尽管前者比后者小60倍。微调过程仅使用了82,000对(指令，输出)数据，比通常训练文本编辑模型从头开始所需的数据要少。

在基础模型的早期，当最强大的模型是商业性的，微调访问有限时，没有很多可供微调的竞争模型。然而，随着开源社区涌现出各种规模、为各种领域量身定制的高质量模型，微调变得更加可行和有吸引力。

不微调的理由

虽然微调可以在多种方式上改进模型，但这些改进中的许多也可以在一定程度上通过其他方式实现。微调可以提高模型的性能，但精心设计的提示和上下文也可以。微调可以帮助结构化输出，但如第2章所讨论的，许多其他技术也可以做到这一点。

首先，虽然为特定任务微调模型可以提高其在该任务上的性能，但可能会降低其在其他任务上的性能。当你打算将这个模型用于预期接收各种提示的应用程序时，这可能会令人沮丧。

想象一下，你需要一个模型处理三种类型的查询：产品推荐、订单更改和一般反馈。最初，该模型在产品推荐和一般反馈方面表现良好，但在订单更改方面表现不佳。为了解决这个问题，你用一个关于订单更改的（查询，响应）对数据集对模型进行微调。微调后的模型在这种类型的查询上表现可能确实更好，但在其他两个任务上表现更差。

在这种情况下你怎么办？你可以在所有关心的查询上微调模型，而不仅仅是订单更改。如果你似乎无法让一个模型在所有任务上都表现良好，可以考虑为不同任务使用单独的模型。如果你希望将这些单独的模型合并为一个以便于服务，你也可以考虑将它们合并在一起，这将在本章后面讨论。

如果你刚开始尝试一个项目，微调很少是你应该首先尝试的事情。微调需要高前期投资和持续维护。首先，你需要数据。手动获取标注数据可能缓慢且昂贵，特别是对于需要批判性思维和领域专业知识的任务。开源数据和AI生成的数据可以降低成本，但它们的有效性高度可变。

其次，微调需要了解如何训练模型。你需要评估基础模型以选择一个进行微调。根据你的需求和资源，选择可能有限。虽然微调框架和API可以自动化实际微调过程中的许多步骤，但你仍然需要了解可以调整的不同训练旋钮，监控学习过程，以及在出现问题时进行调试。例如，你需要了解优化器如何工作，使用什么学习率，需要多少训练数据，如何解决过拟合/欠拟合问题，以及如何在整个过程中评估模型。

第三，一旦有了微调模型，你需要弄清楚如何提供服务。你是自己托管还是使用API服务？如第9章所述，大型模型特别是LLM的推理优化并不简单。如果你已经在内部托管模型并熟悉如何操作模型，微调需要较少的技术飞跃。

更重要的是，你需要建立监控、维护和更新模型的政策和预算。当你迭代微调模型时，新的基础模型正在以惊人的速度开发。这些基础模型可能会比你增强微调模型的速度更快地改进。如果一个新的基础模型在你的特定任务上表现优于你的微调模型，性能提升必须达到多大程度才能让你切换到新的基础模型？如果一个新的基础模型不会立即超过你现有的模型，但在微调后有潜力这样做——你会尝试吗？

在许多情况下，切换到更好的模型只会提供小的增量改进，你的任务可能比具有更大回报的项目（如启用新用例）获得较低的优先级。

AI工程实验应该从提示开始，遵循第6章讨论的最佳实践。只有在单靠提示证明不足时，才探索更先进的解决方案。确保你已经彻底测试了各种提示，因为模型的性能可能会随着不同的提示而有很大变化。

我交谈过的许多从业者都分享了一个类似的故事：有人抱怨提示无效，并坚持要进行微调。经调查，发现提示实验很少且不系统。指令不清晰，示例不代表实际数据，指标定义不良。在完善提示实验过程后，提示质量提高到足以满足他们的应用需求。

微调领域特定任务

当心这种论点：通用模型不能很好地处理领域特定任务，因此你必须为特定任务微调或训练模型。随着通用模型变得更加强大，它们在领域特定任务上也变得更好，可以超过领域特定模型。

一个有趣的早期专业模型是BloombergGPT，由彭博社在2023年3月推出。当时市场上最强大的模型都是专有的，彭博社想要一个在金融任务上表现良好的中等规模模型，可以在内部托管以用于敏感数据用例。这个拥有500亿参数的模型需要130万A100 GPU小时的训练。计算成本估计在130万到260万美元之间，不包括数据成本(Wu等，2023)。

同月，OpenAI发布了GPT-4-0314。李等人(2023)的研究表明，GPT-4-0314在各种金融基准测试中显著优于BloombergGPT。表7-1提供了两个此类基准的详细信息。

表7-1. 像GPT-4这样的通用模型在金融领域可以超过金融模型。

模型	FiQA情感分析(加权F1)	ConvFinQA(准确率)
GPT-4-0314(零样本)	87.15	76.48
BloombergGPT	75.07	43.41

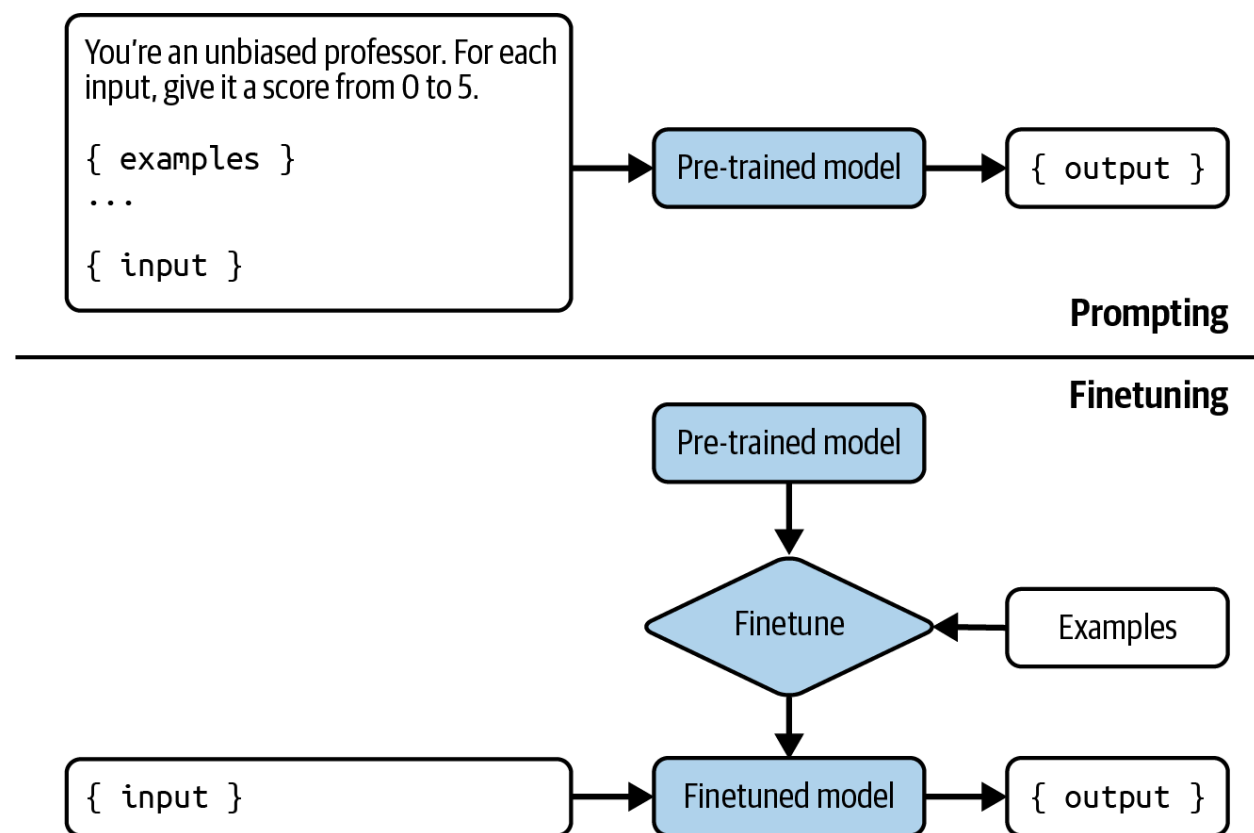
此后，发布了几个性能与GPT-4相当的中等规模模型，包括Claude 3.5 Sonnet(70B参数)、Llama 3-70B-Instruct和Qwen2-72B-Instruct。后两者是开放权重的，可以自行托管。

由于基准测试不足以捕捉现实世界的性能，BloombergGPT可能在彭博社的特定用例中表现良好。彭博团队通过训练这个模型获得了宝贵的经验，这可能使他们能够更好地开发和操作未来的模型。

微调和提示实验都需要系统的过程。进行提示实验使开发人员能够建立评估管道、数据注释指南和实验跟踪实践，这些都是微调的垫脚石。

在引入提示缓存之前，微调的一个好处是它可以帮助优化标记使用。提示中添加的示例越多，模型将使用的输入标记就越多，这增加了延迟和成本。与其在每个提示中包含示例，不如在这些示例上微调模型。这允许你在微调模型时使用更短的提示，如图7-2所示。

通过提示缓存，重复的提示段可以被缓存以重复使用，这不再是一个强大的好处。提示缓存在第9章中有进一步讨论。然而，你可以在提示中使用的示例数量仍然受到最大上下文长度的限制。通过微调，你可以使用的示例数量没有限制。



微调和RAG

一旦你从提示中获得了最大的性能提升，你可能会想知道是否应该进行RAG或微调。答案取决于你的模型失败是基于信息还是基于行为。

如果模型失败是因为缺乏信息，提供相关信息源访问的RAG系统可以帮助。基于信息的失败发生在输出事实错误或过时。以下是发生基于信息失败的两个示例场景：

模型没有信息。公共模型不太可能有私人或组织的信息。当模型没有信息时，它要么告诉你，要么产生幻觉答案。

模型有过时信息。如果你问："Taylor Swift发行了多少张录音室专辑？"正确答案是11，但模型回答10，可能是因为模型的截止日期在最新专辑发行之前。

Ovadia等人(2024)的论文"微调还是检索？"表明，对于需要最新信息的任务，如关于当前事件的问题，RAG的表现优于微调模型。不仅如此，使用基础模型的RAG表现优于使用微调模型的RAG

，如表7-2所示。这一发现表明，虽然微调可以提高模型在特定任务上的性能，但也可能导致其 在其他领域的性能下降。

表7-2. 在Ovadia等人(2024)策划的关于当前事件的问答任务上，RAG优于微调。FT-reg和FT-par指作者使用的两种不同微调方法。

基础模型	基础模型+RAG	FT-reg	FT-par	FT-reg+RAG	FT-par+RAG
Mistral-7B	0.481	0.875	0.504	0.588	0.810
Llama 2-7B	0.353	0.585	0.219	0.392	0.326
Orca 2-7B	0.456	0.876	0.511	0.566	0.820

另一方面，如果模型有行为问题，微调可能会有所帮助。一个行为问题是，模型的输出在事实上是正确的，但与任务无关。例如，你要求模型为软件项目生成技术规范，提供给你的工程团队。虽然生成的规范准确，但缺乏你的团队需要的细节。用定义良好的技术规范微调模型可以使输出更加相关。

另一个问题是模型未能遵循预期的输出格式。例如，如果你要求模型编写HTML代码，但生成的代码无法编译，这可能是因为模型在其训练数据中没有足够接触HTML。你可以通过在微调期间让模型接触更多HTML代码来纠正这一点。

语义解析是一类任务，其成功取决于模型生成预期格式输出的能力，因此通常需要微调。语义解析在第2章和第6章中有简要讨论。提醒一下，语义解析意味着将自然语言转换为结构化格式，如JSON。强大的现成模型通常对常见、复杂性较低的语法如JSON、YAML和正则表达式表现良好。然而，对于互联网上示例较少的语法，如不太流行工具的领域特定语言或复杂语法，它们可能表现不佳。

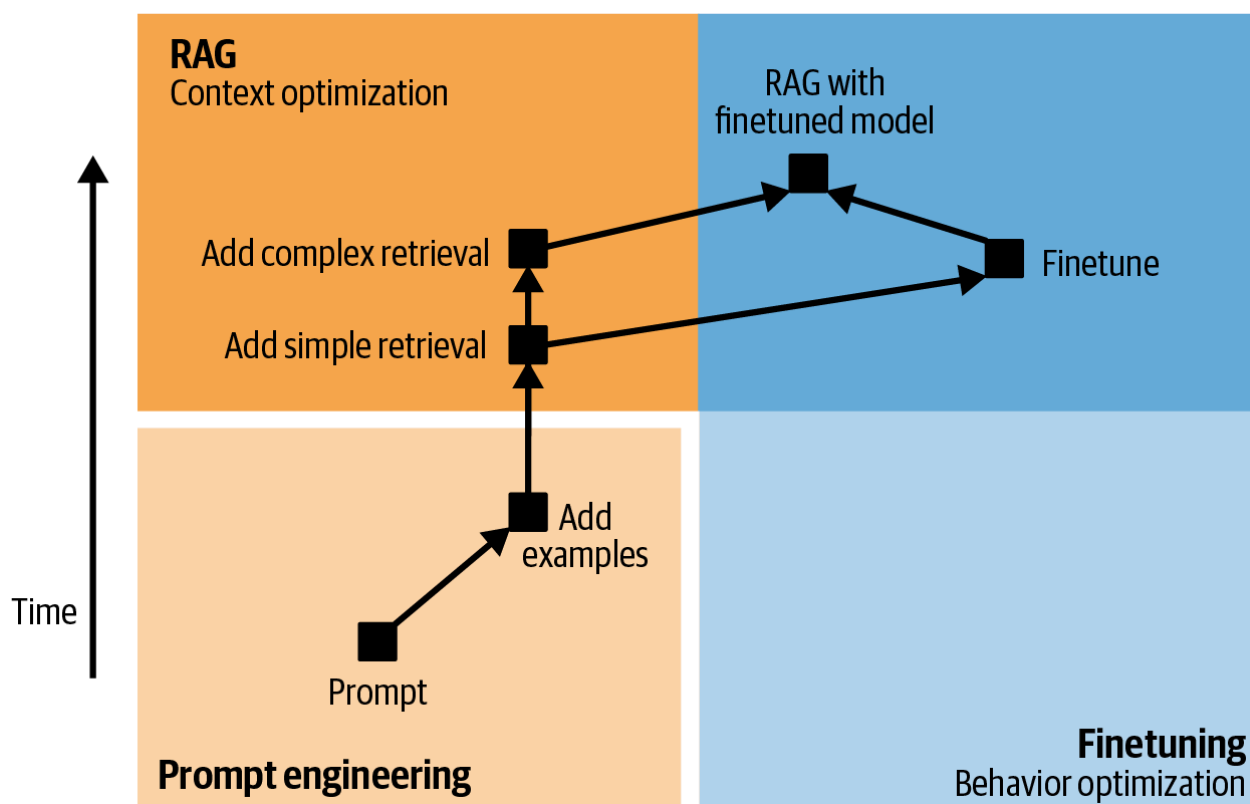
简而言之，微调是为了形式，RAG是为了事实。RAG系统为模型提供外部知识，以构建更准确、更丰富的答案。RAG系统可以帮助减轻模型的幻觉。另一方面，微调帮助模型理解和遵循语法和样式。虽然如果有足够的高质量数据，微调可能减少幻觉，但如果数据质量低，也可能加剧幻觉。

如果你的模型同时存在信息和行为问题，从RAG开始。RAG通常更容易，因为你不必担心策划训练数据或托管微调模型。做RAG时，从简单的基于术语的解决方案(如BM25)开始，而不是直接跳到需要向量数据库的方法。

RAG也可以带来比微调更显著的性能提升。Ovadia等人(2024)表明，对于MMLU基准中几乎所有的问题类别，RAG的表现优于三种不同模型的微调：Mistral 7B、Llama 2-7B和Orca 2-7B。

然而，RAG和微调并不互斥。有时它们可以一起使用，以最大化应用程序的性能。在同一实验中，Ovadia等人(2024)表明，在微调模型之上加入RAG可以在43%的情况下提高其在MMLU基准上的性能。重要的是要注意，在这个实验中，与单独使用RAG相比，在57%的情况下，在微调模型上使用RAG不会提高性能。

对于所有应用程序，没有通用的工作流程。图7-3显示了应用程序开发过程随时间可能遵循的一些路径。箭头表示你可能尝试的下一步。该图受OpenAI(2023)展示的示例工作流程的启发。



因此，在适应模型到任务的工作流程中，请注意，在任何适应步骤之前，你应该定义评估标准并设计评估管道，如第4章所述。这个评估管道是你在开发应用程序时衡量进展的依据。评估不仅仅发生在开始阶段，它应该存在于整个过程的每个步骤中：

1. 尝试仅通过提示让模型执行你的任务。使用第5章中介绍的提示工程最佳实践，包括系统地版本化你的提示。
2. 在提示中添加更多示例。根据用例，所需的示例数量可能在1到50之间。
3. 如果你的模型经常因缺少信息而失败，将其连接到可以提供相关信息的数据源。开始使用RAG时，先使用基本检索方法，如基于术语的搜索。即使使用简单的检索，添加相关和准确的知识应该会导致模型性能的一些改进。

4. 根据模型的失败模式，你可能会探索以下一个步骤：
 - a. 如果模型继续有基于信息的失败，你可能想尝试更高级的RAG方法，如基于嵌入的检索。
 - b. 如果模型继续有行为问题，例如它继续生成不相关、格式错误或不安全的响应，你可以选择微调。基于嵌入的检索通过在管道中引入额外组件增加了推理复杂性，而微调增加了模型开发的复杂性，但推理保持不变。
5. 结合RAG和微调以获得更大的性能提升。

如果在考虑了微调和其他替代技术的所有利弊后，你决定微调你的模型，本章的其余部分将为你提供帮助。首先，让我们看看微调的首要挑战：内存瓶颈。

内存瓶颈

由于微调需要大量内存，许多微调技术旨在最小化其内存占用。理解这种内存瓶颈的原因对于理解这些技术为何以及如何工作是必要的。这种理解反过来可以帮助你选择最适合你的微调方法。

除了解释微调的内存瓶颈外，本节还介绍了用于粗略计算每个模型内存使用的公式。这种计算对于估计你需要什么硬件来服务或微调模型很有用。

因为内存计算需要分解底层ML和计算概念，所以本节技术性较强。如果你已经熟悉这些概念，可以跳过它们。

理解内存瓶颈的关键要点

如果你决定跳过本节，以下是一些关键点。如果你发现这些要点中的任何一点不熟悉，本节中的概念应该有助于解释：

1. 由于基础模型的规模，内存是处理它们的瓶颈，无论是推理还是微调。微调所需的内存通常比推理所需的内存高得多，这是因为神经网络训练的方式。
2. 模型在微调期间内存占用的主要因素是其参数数量、可训练参数数量和数值表示。
3. 可训练参数越多，内存占用越高。通过减少可训练参数的数量，可以减少微调所需的内存。减少可训练参数的数量是PEFT(参数高效微调)的动机。
4. 量化是指将模型从更多位的格式转换为更少位的格式的做法。量化是减少模型内存占用的直接有效方式。对于一个130亿参数的模型，使用FP32意味着每个权重4字节或整个权重52GB。如果可以将每个值减少到只有2字节，模型权重所需的内存减少到26GB。

5. 推理通常使用尽可能少的位, 如16位、8位甚至4位。
6. 训练对数值精度更敏感, 因此在较低精度下训练模型更难。训练通常以混合精度进行, 一些操作以较高精度(例如32位)进行, 一些以较低精度(例如16位或8位)进行。

反向传播和可训练参数

决定模型在微调期间内存占用的关键因素是其可训练参数的数量。可训练参数是在微调期间可以更新的参数。在预训练期间, 所有模型参数都会更新。在推理期间, 没有模型参数会更新。在微调期间, 部分或全部模型参数可能会更新。保持不变的参数是冻结参数。

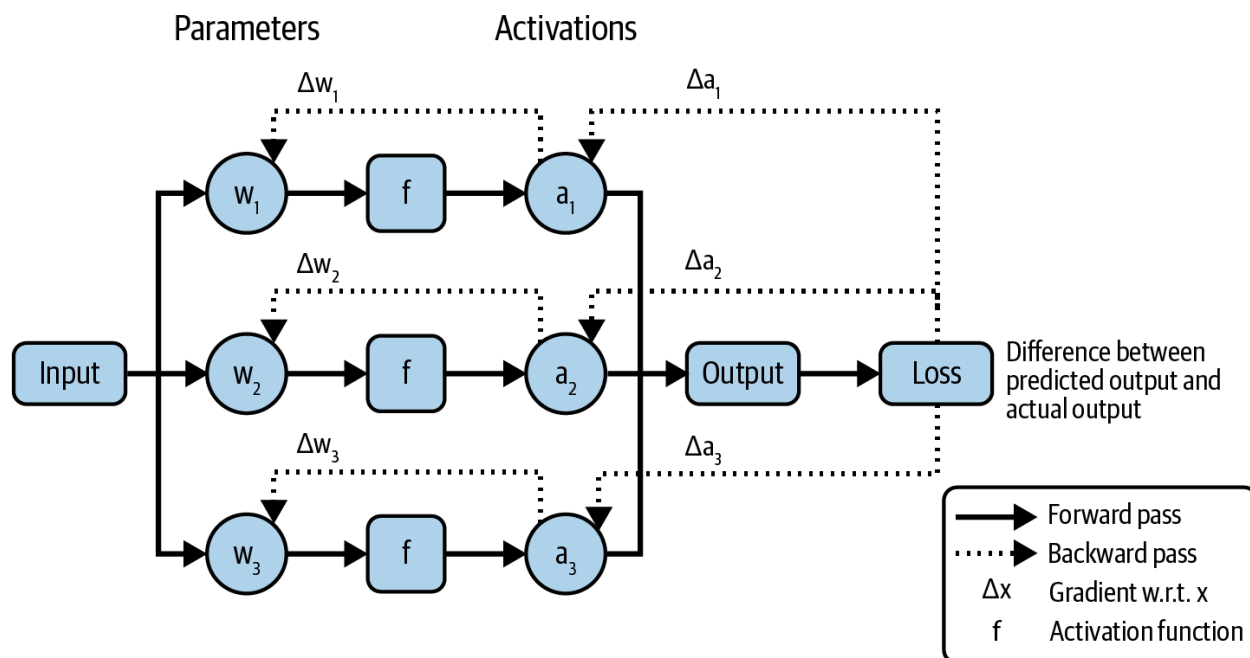
每个可训练参数所需的内存是由模型训练的方式决定的。截至目前, 神经网络通常使用称为反向传播的机制进行训练。使用反向传播, 每个训练步骤包括两个阶段:

1. 前向传递: 从输入计算输出的过程。
2. 反向传递: 使用前向传递中聚合的信号更新模型权重的过程。

在推理期间, 只执行前向传递。在训练期间, 两个传递都会执行。在高层次上, 反向传递的工作原理如下:

1. 将前向传递计算的输出与预期输出(真实值)进行比较。如果它们不同, 模型犯了错误, 参数需要调整。计算输出与预期输出之间的差异称为损失。
2. 计算每个可训练参数对错误的贡献程度。这个值称为梯度。在数学上, 梯度是通过对每个可训练参数的损失求导计算的。每个可训练参数都有一个梯度值。如果一个参数有高梯度, 它对损失的贡献很大, 应该调整更多。
3. 使用相应的梯度调整可训练参数值。给定其梯度值, 每个参数应该如何重新调整, 由优化器决定。常见的优化器包括SGD(随机梯度下降)和Adam。对于基于Transformer的模型, Adam是目前最广泛使用的优化器。

图7-4显示了一个假设的神经网络的前向和反向传递, 该网络有三个参数和一个非线性激活函数。我使用这个虚拟神经网络来简化可视化。



在反向传递期间，每个可训练参数都带有额外的值，其梯度和优化器状态。因此，可训练参数越多，需要存储这些额外值的内存就越多。

内存计算

了解模型需要多少内存很有用，这样你就可以为它使用正确的硬件。通常，你可能已经有了硬件，需要计算是否能够负担运行某个模型。如果一个模型需要30GB的内存进行推理，一个有24GB内存的芯片将不够用。

模型的内存占用取决于模型以及工作负载和用于减少内存使用的不同优化技术。因为不可能考虑所有的优化技术和工作负载，在本节中，我只概述了近似计算的公式，这应该给你一个粗略的想法，了解操作模型需要多少内存，无论是在推理还是训练期间。

注意

推理和训练有不同的内存配置文件是训练和推理芯片分化的原因之一，如第9章所述。

推理所需的内存

在推理期间，只执行前向传递。前向传递需要模型权重的内存。设 N 为模型的参数数量， M 为每个参数所需的内存；加载模型参数所需的内存为：

$$N \times M$$

前向传递还需要激活值的内存。Transformer模型需要注意力机制的键值向量的内存。激活值和键值向量的内存随序列长度和批处理大小线性增长。

对于许多应用，激活和键值向量的内存可以假定为模型权重内存的20%。如果你的应用使用更长的上下文或更大的批处理大小，实际所需的内存会更高。这个假设使模型的内存占用为：

$$N \times M \times 1.2$$

考虑一个130亿参数的模型。如果每个参数需要2字节，模型的权重将需要130亿 $\times$ 2字节 = 26GB。推理的总内存将是26GB $\times$ 1.2 = 31.2GB。

模型的内存占用随其大小迅速增长。随着模型变得更大，内存成为操作它们的瓶颈。一个700亿参数的模型，每个参数2字节，仅其权重就需要惊人的140GB内存。

训练所需的内存

训练模型需要模型权重和激活的内存，这已经讨论过。此外，你需要梯度和优化器状态的内存，这与可训练参数的数量成比例。

总体而言，训练所需的内存计算为：

- 训练内存 = 模型权重 + 激活 + 梯度 + 优化器状态

提示

在反向传递期间，每个可训练参数需要一个梯度值加上零到两个优化器状态值，取决于优化器：

- 一个普通的SGD优化器没有状态。
- 一个动量优化器为每个可训练参数存储一个值。
- 一个Adam优化器为每个可训练参数存储两个值。

想象一下，你正在使用Adam优化器更新一个130亿参数模型的所有参数。因为每个可训练参数有三个值用于其梯度和优化器状态，如果存储每个值需要两个字节，梯度和优化器状态所需的内存将是：

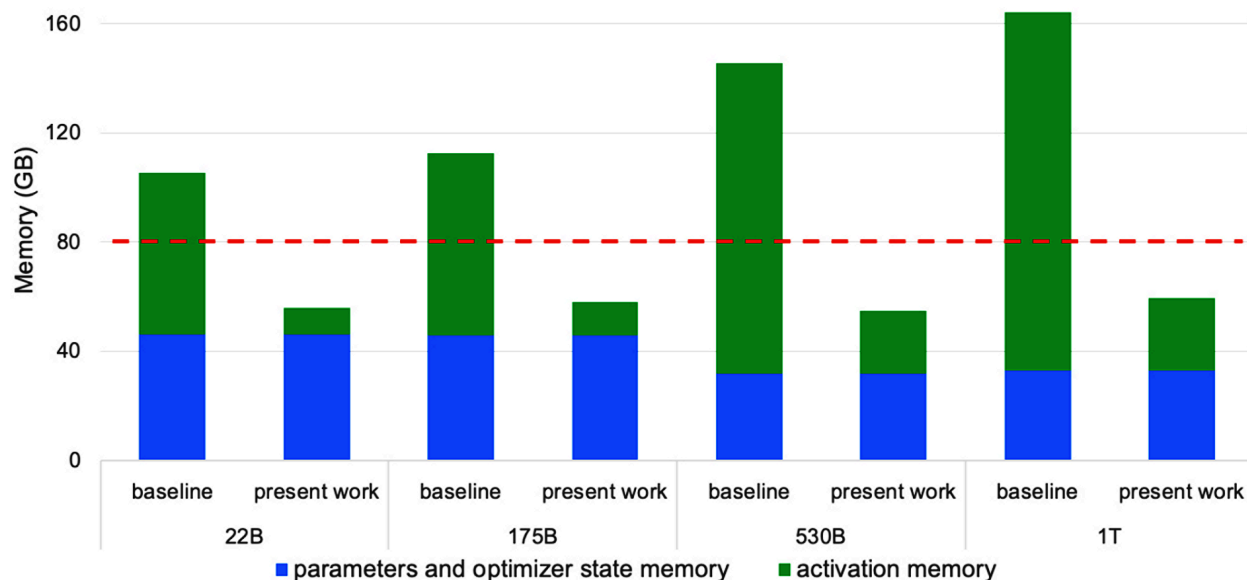
$$130\text{亿} \times 3 \times 2\text{字节} = 78\text{GB}$$

然而，如果你只有10亿可训练参数，梯度和优化器状态所需的内存将只有：

$$10\text{亿} \times 3 \times 2\text{字节} = 6\text{GB}$$

需要注意的重要一点是，在前面的公式中，我假设激活所需的内存少于模型权重所需的内存。然而，实际上，激活内存可能大得多。如果存储激活用于梯度计算，激活所需的内存可能远远超过模型权重所需的内存。图7-5显示了不同规模的Megatron模型中激活所需的内存与模型权重所需的内存相比较，根据Korthikanti等人(2022)的论文“减少大型Transformer模型中的激活重新计算”。

减少激活所需内存的一种方法是不存储它们。与其存储激活以便重用，不如在必要时重新计算激活。这种技术称为梯度检查点或激活重新计算。虽然这减少了内存需求，但由于重新计算，它增加了训练所需的时间。



数值表示

在到目前为止的内存计算中，我假设每个值占用两个字节的内存。在模型中表示每个值所需的内存直接影响模型的整体内存占用。如果将每个值所需的内存减少一半，模型权重所需的内存也会减少一半。

在讨论如何减少每个值所需的内存之前，了解数值表示很有用。神经网络中的数值传统上表示为浮点数。最常见的浮点格式系列是FP系列，它遵循电气和电子工程师协会(IEEE)浮点算术标准(IEEE 754)：

- FP32使用32位(4字节)表示浮点数。这种格式称为单精度。
- FP64使用64位(8字节)，称为双精度。
- FP16使用16位(2字节)，称为半精度。

虽然FP64仍在许多计算中使用——截至本书撰写时，FP64是NumPy和pandas的默认格式——但由于其内存占用，它在神经网络中很少使用。FP32和FP16更常见。AI工作负载中其他流行的浮

点格式包括BF16(BFloat16)和TF32(TensorFloat-32)。BF16由谷歌设计, 用于优化TPU上的AI性能, TF32由NVIDIA为GPU设计。

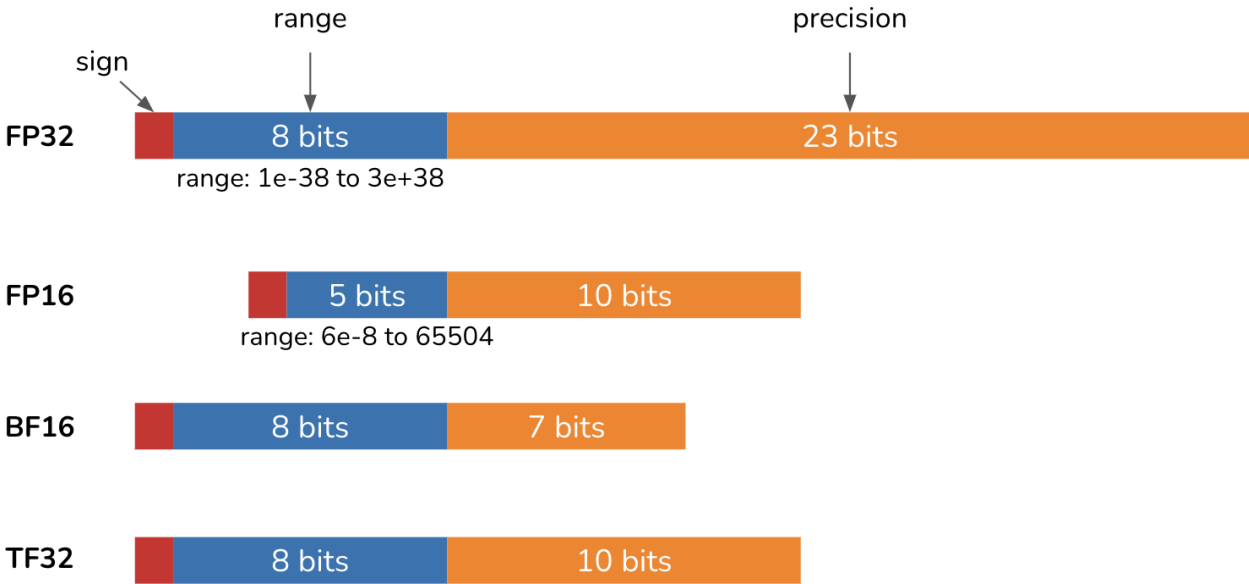
数值也可以表示为整数。虽然目前不如浮点格式常见, 但整数表示越来越流行。常见的整数格式有INT8(8位整数)和INT4(4位整数)。

每种浮点格式通常有1位表示数字的符号, 即负或正。其余位在范围和精度之间分配:

范围 范围位的数量决定了格式可以表示的值的范围。位数越多, 范围越广。这类似于拥有更多位数可以表示更广泛的数字范围。

精度 精度位的数量决定了数字表示的精确程度。减少精度位数使数字精度降低。例如, 如果将10.1234转换为只支持两位小数的格式, 这个值变为10.12, 比原始值精度低。

图7-6显示了不同的浮点格式及其范围和精度位。



位数更多的格式被认为是更高精度的。将高精度格式的数字转换为低精度格式(例如, 从FP32到FP16)意味着降低其精度。降低精度可能导致值变化或产生误差。表7-3显示了FP32值如何转换为FP16、BF16和TF32。

表7-3. 将FP32值转换为低精度格式。结果不准确之处以斜体显示。

FP32	FP16	BF16	TF32
------	------	------	------

0.0123456789	0.0123443603515625	0.0123291	0.0123443603515625
0.123456789	0.12347412109375	0.123535	0.1234130859375
1.23456789	1.234375	1.23438	1.234375
12.3456789	12.34375	12.375	12.34375
123.456789	123.4375	123.5	123.4375
1234.56789	1235.0	1232.0	1234.0
12345.6789	12344.0	12352.0	12344.0
123456.789	INF ^a	123392.0	123456.0
1234567.89	INF	1236990.0	1233920.0

^a FP16中超出范围的值被四舍五入为无穷大。

请注意表7-3中，尽管BF16和FP16有相同数量的位，但BF16有更多的位用于范围，更少的位用于精度。这使BF16能够表示超出FP16范围的大值。然而，这也使BF16比FP16精度低。例如，1234.56789在FP16中是1235.0(值变化0.035%)，但在BF16中是1232.0(值变化0.208%)。

警告

使用模型时，确保以其设计的格式加载模型。将模型加载到错误的数值格式可能导致模型发生重大变化。例如，Llama 2在发布时将其权重设置为BF16。然而，许多团队在FP16中加载模型，随后沮丧地发现模型的质量比宣传的差得多。虽然这种误解浪费了很多人的时间，但好处是它迫使许多人了解数值表示。

适合你的正确格式取决于工作负载的数值分布(如你需要的值范围)，工作负载对小数值变化的敏感程度，以及底层硬件。

量化

表示模型值所需的位越少，模型的内存占用就越低。一个100亿参数的模型采用32位格式需要40GB的权重，但同一模型采用16位格式只需要20GB。降低精度，也称为量化，是减少模型内存占用的一种廉价且极其有效的方法。它实施简单，可以在任务和架构上推广。在ML环境中，低精度通常指任何比标准FP32位数更少的格式。

量化与降低精度

严格来说，只有当目标格式是整数时，才是量化。然而，在实践中，量化用于指代所有将值转换为低精度格式的技术。在本书中，我使用量化来指代精度降低，以与文献保持一致。

进行量化时，你需要决定量化什么以及何时量化：

量化什么 理想情况下，你想量化消耗大部分内存的东西，但也取决于你可以在不过度影响性能的情况下量化什么。如“内存计算”中所述，模型在推理期间内存占用的主要因素是模型的权重和激活。权重量化比激活量化更常见，因为权重激活往往对性能有更稳定的影响，精度损失更小。

何时量化 量化可以在训练期间或训练后发生。训练后量化(PTQ)意味着在模型完全训练后对其进行量化。PTQ是迄今为止最常见的。它对通常不训练模型的AI应用程序开发人员更相关。

推理量化

在深度学习的早期，使用32位的FP32训练和服务模型是标准。自2010年代后期以来，用16位甚至更低精度提供模型变得越来越普遍。例如，Dettmers等人(2022)通过LLM.int8()将LLM量化到8位，通过QLoRA(Dettmers等人，2023)量化到4位，做了出色的工作。

模型也可以以混合精度提供服务，即在可能的情况下降低值的精度，在必要时保持较高精度。为了在设备上提供模型服务，苹果(2024)利用了一种量化方案，使用2位和4位格式的混合，平均每权重3.5位。同样在2024年，为了预测4位神经网络的发展，NVIDIA宣布了他们的新GPU架构Blackwell，支持4位浮点的模型推理。

一旦进入8位及以下，数值表示变得更加棘手。你可以将参数值保持为浮点数，使用一种迷你浮点格式，如FP8(8位)和FP4(4位)。然而，更常见的是，参数值被转换为整数格式，如INT8或INT4。

量化是有效的，但它可以发展到何种程度是有限制的。每个值不能少于1位，有些人尝试了1位表示，例如BinaryConnect(Courbariaux等人，2015)、Xnor-Net(Rastegari等人，2016)和BitNet(Wang等人，2023)。

2024年，微软研究人员(Ma等人)宣布我们正在进入1位LLM时代，引入了BitNet b1.58，这是一种基于Transformer的语言模型，每个参数只需要1.58位，其性能与16位Llama 2(Touvron等人，2023)在高达39亿参数时相当，如表7-4所示。

表7-4. BitNet b1.58的性能与Llama 2 16位在不同基准测试和不同模型大小(最高39亿参数)上的比较。结果来自Ma等人(2024)。

模型	大小	ARCe	ARCc	HS	BQ	OQ	PQ	WGe	平均
Llama LLM	700M	54.7	23.0	37.0	60.0	20.2	68.9	54.8	45.5
BitNet b1.58	700M	51.8	21.4	35.1	58.2	20.0	68.1	55.2	44.3
Llama LLM	1.3B	56.9	23.5	38.5	59.1	21.6	70.0	53.9	46.2
BitNet b1.58	1.3B	54.9	24.2	37.7	56.7	19.6	68.8	55.8	45.4
Llama LLM	3B	62.1	25.6	43.3	61.8	24.6	72.1	58.2	49.7
BitNet b1.58	3B	61.4	28.3	42.9	61.5	26.6	71.5	59.3	50.2
BitNet b1.58	3.9B	64.2	28.7	44.2	63.5	24.2	73.2	60.5	51.2

降低精度不仅减少了内存占用，而且通常还能提高计算速度。首先，它允许更大的批处理大小，使模型能够并行处理更多输入。其次，降低精度加速了计算，进一步减少了推理延迟和训练时间。为了说明这一点，考虑两个数字的加法。如果我们逐位执行加法，每一位需要 t 纳秒，32位将需要 $32t$ 纳秒，而16位只需要 $16t$ 纳秒。然而，由于格式转换所需的额外计算，降低精度并不总是减少延迟。

降低精度也有缺点。每次转换通常会导致小的值变化，许多小变化可能导致大的性能变化。如果一个值超出了降低精度格式可以表示的范围，它可能被转换为无穷大或任意值，导致模型质量进一步下降。如何在对模型性能影响最小的情况下降低精度是一个活跃的研究领域，由模型开发者以及硬件制造商和应用程序开发者追求。

较低精度的推理已成为标准。模型使用较高精度格式训练以最大化性能，然后降低其精度用于推理。主要的ML框架，包括PyTorch、TensorFlow和Hugging Face的transformers，都提供免费的PTQ，只需几行代码。

一些边缘设备仅支持量化推理。因此，用于设备上推理的框架，如TensorFlow Lite和PyTorch Mobile，也提供PTQ。

训练量化

训练期间的量化尚未像PTQ那样普遍，但正在获得动力。训练量化有两个不同的目标：

1. 生产一个在低精度推理期间表现良好的模型。这是为了解决模型质量可能在训练后量化期间下降的挑战。
2. 减少训练时间和成本。量化减少了模型的内存占用，使模型可以在更便宜的硬件上训练，或者在同样的硬件上训练更大的模型。量化还加速了计算，进一步降低了成本。

一种量化技术可能有助于实现这些目标中的一个或两个。

量化感知训练(QAT)旨在创建一个在低精度推理中具有高质量的模型。使用QAT，模型在训练期间模拟低精度(例如8位)行为，这使模型能够学习在低精度下产生高质量输出。然而，QAT不会减少模型的训练时间，因为其计算仍然以高精度执行。由于模拟低精度行为的额外工作，QAT甚至可能增加训练时间。

另一方面，直接在较低精度下训练模型可以帮助实现这两个目标。人们早在2016年就尝试在降低精度下训练模型；见Hubara等人(2016)和Jacob等人(2017)。Character.AI(2024)分享说，他们能够完全在INT8中训练他们的模型，这有助于消除训练/服务精度不匹配，同时也显著提高了训练效率。然而，在较低精度下训练更难，因为反向传播对低精度更敏感。

低精度训练通常以混合精度进行，保留权重的高精度副本，但其他值，如梯度和激活，以低精度保存。你也可以让较不敏感的权重值以较低精度计算，较敏感的权重值以较高精度计算。例如，LLM-QAT(Liu等人，2023)将权重和激活量化为4位，但保持嵌入为16位。

模型应该采用较低精度的部分可以使用许多ML框架提供的自动混合精度(AMP)功能自动设置。

也可能有不同阶段的训练采用不同的精度水平。例如,模型可以在较高精度下训练,但在较低精度下微调。这在基础模型中尤为常见,从头开始训练模型的团队可能是有足够计算能力进行高精度训练的组织。一旦模型发布,拥有较少计算访问权限的开发人员可以在较低精度下微调该模型。

微调技术

我希望前一节已经清楚地说明了为什么微调大规模模型如此内存密集。微调需要的内存越多,能够负担得起微调的人就越少。减少模型内存占用的技术使微调更易获取,让更多人能够适应模型到他们的应用。本节着重于内存高效的微调技术,围绕参数高效微调。

我还将介绍模型合并,这是一种令人兴奋但更具实验性的创建自定义模型的方法。虽然模型合并通常不被视为微调,但我将它包含在本节中,因为它与微调互补。微调使一个模型适应特定需求。模型合并结合多个模型(通常是微调模型)以达到相同目的。

虽然组合多个模型不是一个新概念,但新类型的模型和微调技术激发了许多创造性的模型合并技术,使本节写作特别有微调的几种变体。使得本节的写作特别有趣。

参数高效微调

在微调的早期,模型足够小,人们可以微调整个模型。这种方法称为完全微调。在完全微调中,可训练参数的数量与参数的总数量完全相同。

完全微调看起来可能与训练相似。主要区别在于训练从随机化的模型权重开始,而微调从之前已经训练过的模型权重开始。

如"内存计算"中所述,可训练参数越多,需要的内存就越多。考虑一个70亿参数的模型:

- 如果你使用16位格式如FP16,仅加载模型的权重就需要14GB内存。
- 使用Adam优化器对这个模型进行完全微调,同样使用16位格式,需要额外的70亿 $\times 3 \times 2$ 字节 = 42GB内存。
- 模型权重、梯度和优化器状态所需的总内存为14GB + 42GB = 56GB。

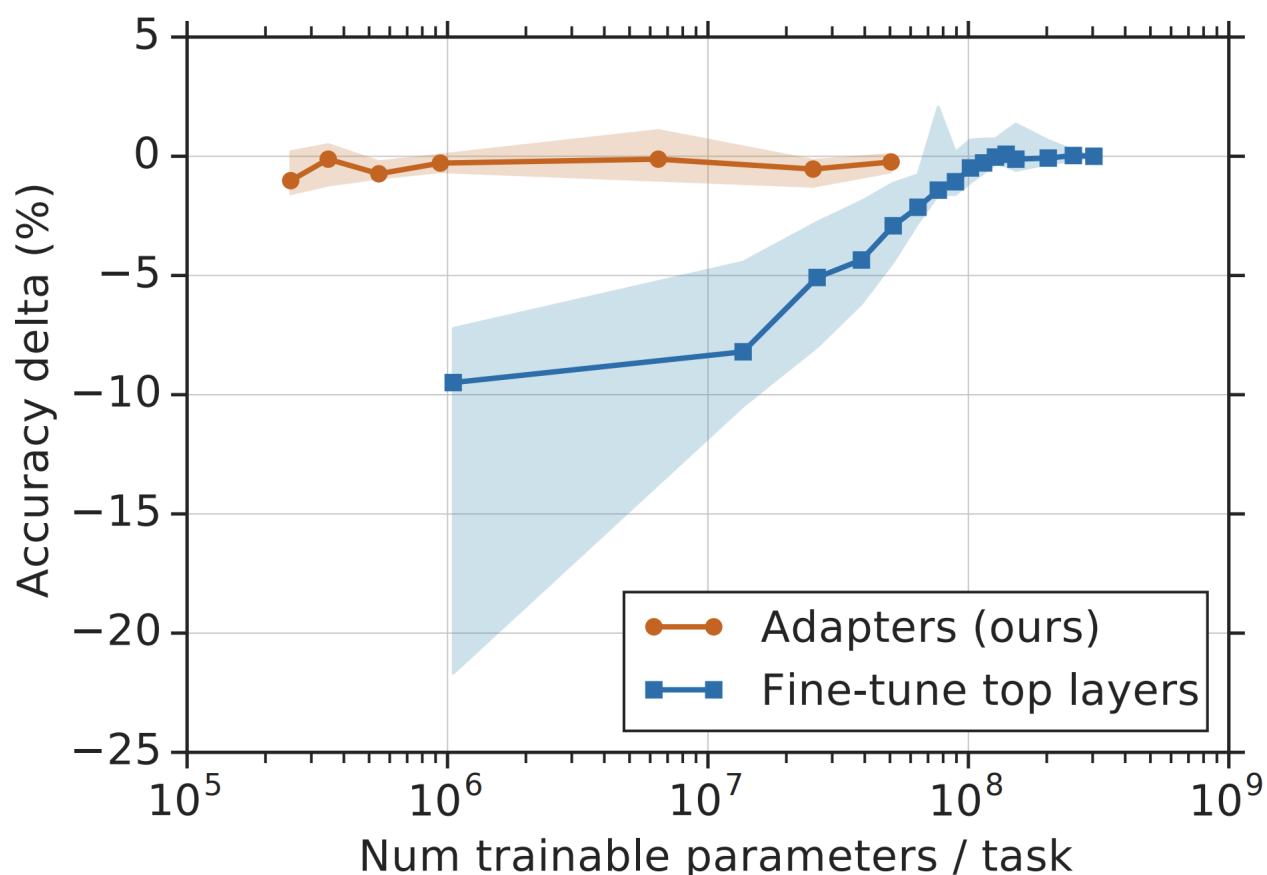
56GB超过了大多数消费级GPU的内存容量,这些GPU通常配备12-24GB内存,高端GPU提供最多48GB。而且这个内存估计还没有考虑激活所需的内存。

注意

要使模型适合特定硬件，你可以减少模型的内存占用或找到更有效使用硬件内存的方法。量化和PEFT等技术可以最小化总内存占用。专注于更好地利用硬件内存的技术包括CPU卸载。你可以将多余的内存卸载到CPU上，而不是尝试将整个模型放在GPU上，如DeepSpeed (Rasley等人, 2020)所示。

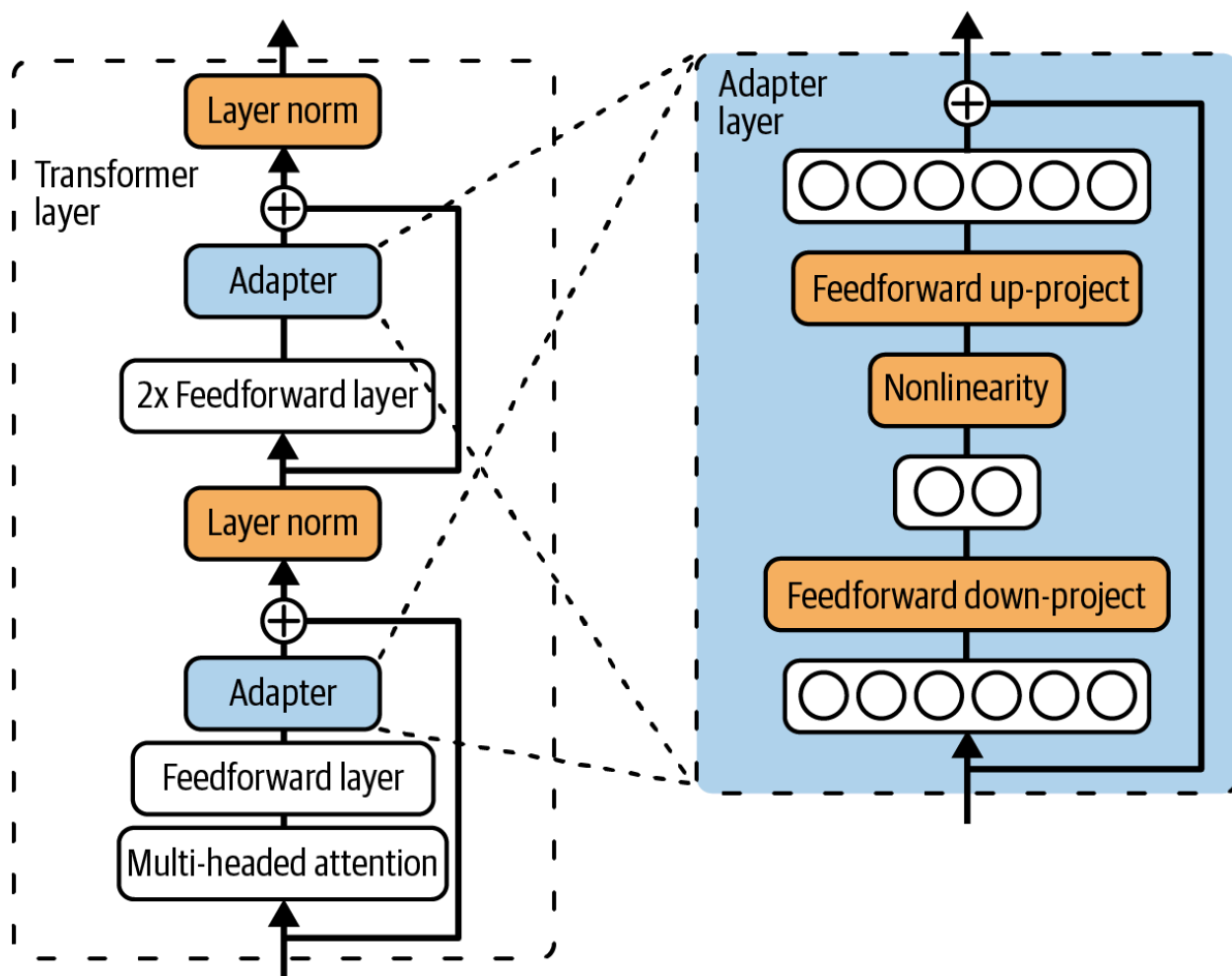
我们还没有涉及到完全微调，特别是监督微调 and 偏好微调，通常需要大量高质量标注数据，这是大多数人无法负担的。由于完全微调的高内存和数据要求，人们开始进行部分微调。在部分微调中，只更新模型的部分参数。例如，如果一个模型有十层，你可能会冻结前九层，只微调最后一层，将可训练参数的数量减少到完全微调的10%。

虽然部分微调可以减少内存占用，但它的参数效率不高。部分微调需要许多可训练参数才能达到接近完全微调的性能。Houlsby等人(2019)的研究表明，对于BERT large (Devlin等人, 2018)，你需要更新大约25%的参数才能在GLUE基准测试 (Wang等人, 2018)上达到与完全微调相当的性能。图7-7显示了不同可训练参数数量的部分微调的性能曲线。



这引出了一个问题：如何使用显著更少的可训练参数达到接近完全微调的性能？从这个探索中产生的微调技术是参数高效的。对于微调方法要被视为参数高效，没有明确的阈值。然而，一般来说，如果一种技术可以使用比完全微调少几个数量级的可训练参数，同时达到接近完全微调的性能，那么它被认为是参数高效的。

PEFT(参数高效微调)的概念由Houlsby等人(2019)引入。作者表明,通过在适当位置向模型插入额外参数,可以使用少量可训练参数达到强大的微调性能。他们在BERT模型的每个transformer块中插入了两个适配器模块,如图7-8所示。



在微调期间,他们保持模型的原始参数不变,只更新适配器。可训练参数的数量是适配器中的参数数量。在GLUE基准测试上,他们使用仅3%的可训练参数数量,达到了与完全微调相差0.4%的性能。图7-7中的橙色线显示了完全微调与使用不同适配器大小进行微调之间的性能差异。

然而,这种方法的缺点是它增加了微调模型的推理延迟。适配器引入了额外的层,这为前向传递增加了更多的计算步骤,减慢了推理速度。

PEFT使得在更经济的硬件上进行微调成为可能,使其对更多开发人员易于获取。PEFT方法通常不仅参数高效,而且样本高效。虽然完全微调可能需要数万到数百万个例子才能实现显著的质量改进,但某些PEFT方法仅需几千个例子就能提供强大的性能。

鉴于PEFT的明显吸引力,PEFT技术正在迅速发展。下一节将对这些技术进行概述,然后深入探讨最常见的PEFT技术:LoRA。

PEFT技术

现有丰富的PEFT世界通常分为两类：基于适配器的方法和基于软提示的方法。然而，未来可能会引入更多新类别。

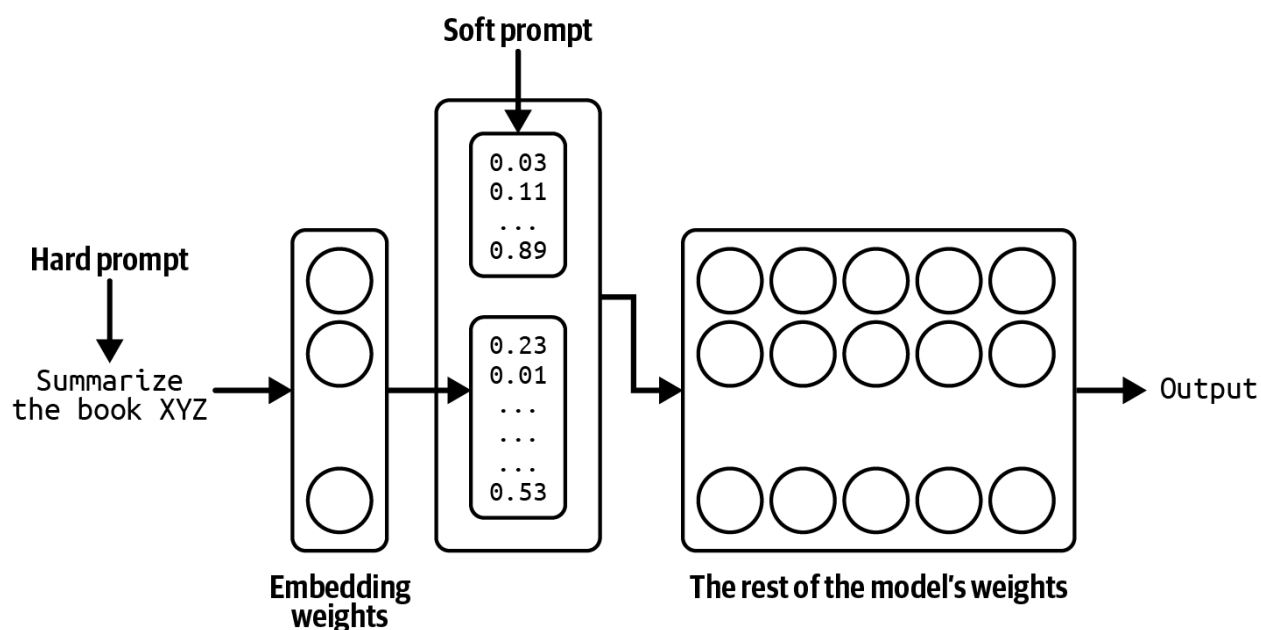
基于适配器的方法是指所有涉及向模型权重添加额外模块的方法，如Houlsby等人(2019)开发的方法。因为基于适配器的方法涉及添加参数，它们也被称为加性方法。

截至本书撰写时，LoRA(Hu等人，2021)是迄今为止最流行的基于适配器的方法，它将是下一节的主题。其他基于适配器的方法包括BitFit(Zaken等人，2021)，它与LoRA大约同时出现。较新的适配器方法包括IA3(Liu等人，2022)，其高效的混合任务批处理策略使其特别适合多任务微调。它已被证明在某些情况下优于LoRA甚至完全微调。LongLoRA(Chen等人，2023)是LoRA的一个变体，它结合了注意力修改技术来扩展上下文长度。

如果基于适配器的方法向模型架构添加可训练参数，基于软提示的方法通过引入特殊的可训练标记来修改模型处理输入的方式。这些额外的标记与输入标记一起被输入到模型中。它们被称为软提示，因为与输入(硬提示)一样，软提示也指导模型的行为。然而，软提示与硬提示在两个方面不同：

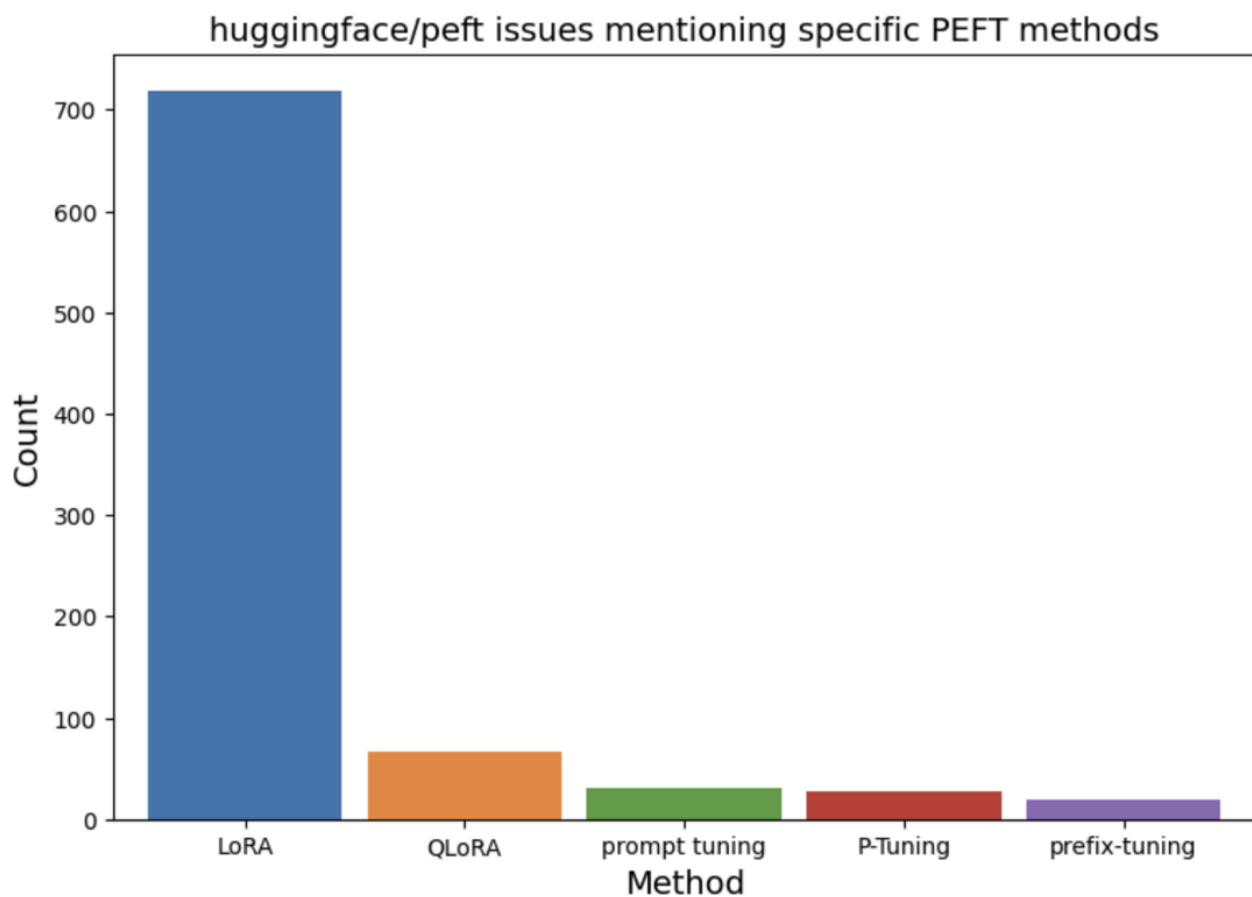
1. 硬提示是人类可读的。它们通常包含离散标记，如"I"、"write"、"a"和"lot"。相比之下，软提示是连续向量，类似于嵌入向量，不是人类可读的。
2. 硬提示是静态的，不可训练的，而软提示可以通过调整过程中的反向传播进行优化，允许它们针对特定任务进行调整。

有些人将软提示描述为提示工程和微调之间的交叉。图7-9展示了如何将软提示与硬提示结合使用来指导模型的行为。



作为一个子领域，软提示调整由一系列名称相似的技术组成，可能令人困惑，如前缀调整(Li和Liang, 2021)、P-Tuning(Liu等人, 2021)和提示调整(Lester等人, 2021)。它们的主要区别在于插入软提示的位置。例如，前缀调整在每个transformer层的输入处添加软提示标记，而提示调整仅在嵌入输入处添加软提示标记。如果你想使用其中任何一种，许多PEFT框架都会为你提供开箱即用的实现。

为了了解正在使用的PEFT方法，我在2024年10月分析了GitHub存储库huggingface/peft上的1000多个开放问题。假设如果有人使用某种技术，他们更有可能报告问题或提出问题。图7-10显示了结果。对于"P-Tuning"，我搜索了关键词"p_tuning"和"p tuning"以考虑不同的拼写。



从这个分析中可以清楚地看出，LoRA占主导地位。软提示不太常见，但似乎对于那些想要比提示工程提供的更多定制，但又不想投资于微调的人来说，正在引起越来越多的兴趣。

由于LoRA的受欢迎程度，下一节将重点关注LoRA如何工作以及它如何解决早期基于适配器的方法提出的挑战。即使你不使用LoRA，这种深入探讨也应该为你探索其他微调方法提供框架。

LoRA

与Houlsby等人(2019)的原始适配器方法不同，LoRA(低秩适应) (Hu等人, 2021)以不会产生额外推理延迟的方式引入额外参数。LoRA不是向基础模型引入额外层，而是使用可以合并回原始层的模块。

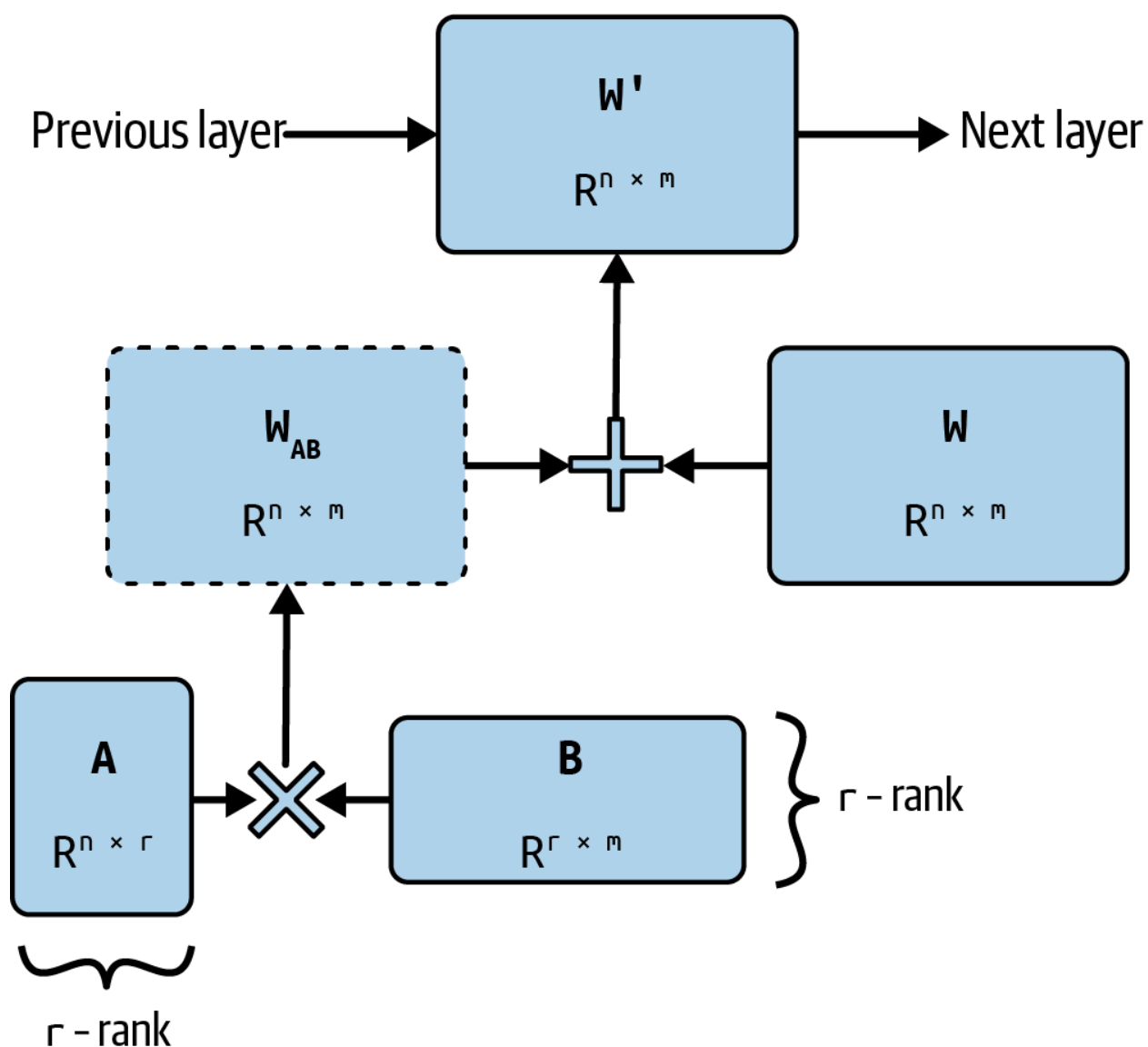
你可以将LoRA应用于单独的权重矩阵。给定一个权重矩阵，LoRA将这个矩阵分解为两个较小矩阵的乘积，然后在将它们合并回原始矩阵之前更新这两个较小的矩阵。

考虑维度为 $n \times m$ 的权重矩阵 W 。LoRA的工作原理如下：

1. 首先，选择较小矩阵的维度。设 r 为所选值。构造两个矩阵： A (维度 $n \times r$)和 B (维度 $r \times m$)。它们的乘积是 WAB ，与 W 的维度相同。 r 是LoRA的秩。

2. 将 WAB 添加到原始权重矩阵 W 中，创建一个新的权重矩阵 W' 。使用 W' 代替 W 作为模型的一部分。你可以使用超参数 α 来确定 WAB 应该对新矩阵贡献多少： $W' = W + \alpha \cdot WAB$
3. 在微调期间，只更新 A 和 B 中的参数。 W 保持不变。

图7-11展示了这个过程。



注意

LoRA(低秩适应)建立在低秩分解的概念上，这是一种长期存在的降维技术。关键思想是你可以将一个大矩阵分解为两个较小矩阵的乘积，以减少参数数量，从而减少

计算和内存需求。例如，一个 9×9 的矩阵可以被分解为维度为 9×1 和 1×9 的两个矩阵的乘积。原始矩阵有81个参数，但两个乘积矩阵合起来只有18个参数。

第一个分解矩阵的列数和第二个分解矩阵的列数对应于分解的秩。原始矩阵是满秩的，而两个较小的矩阵代表一个低秩近似。

虽然分解可以显著减少参数数量，但它是有损的，因为它只近似原始矩阵。秩越高，分解可以保留的原始矩阵信息就越多。

与原始适配器方法一样，LoRA具有参数高效和样本高效的特点。分解使LoRA能够使用更少的可训练参数。LoRA论文显示，对于GPT-3，LoRA在几个任务上达到了与完全微调相当或更好的性能，同时只使用~4.7M可训练参数，仅为完全微调的0.0027%。

为什么**LoRA**有效？

像LoRA这样的参数高效方法已经变得如此受欢迎，以至于许多人认为它们理所当然。但为什么参数效率是可能的呢？如果一个模型在预训练期间需要大量参数来学习某些行为，难道它不应该需要大量参数来在微调期间改变它的行为吗？

同样的问题也可以针对数据提出。如果一个模型需要大量数据来学习一种行为，难道它不应该需要大量数据来有意义地改变这种行为吗？怎么可能需要数百万或数十亿个例子来预训练一个模型，但只需几百或几千个例子来微调它？

许多论文认为，虽然LLM有很多参数，但它们的内在维度非常低；参见Li等人(2018)；Aghajanyan等人(2020)；和Hu等人(2021)。他们表明，预训练隐式地最小化了模型的内在维度。令人惊讶的是，较大的模型在预训练后往往具有较低的内在维度。这表明预训练充当了下游任务的压缩框架。换句话说，LLM训练得越好，使用少量可训练参数和少量数据微调模型就越容易。

你可能想知道，如果低秩分解效果这么好，为什么我们不在预训练中也使用LoRA？与其预训练一个大模型，然后只在微调期间应用低秩分解，我们是否可以从一开始就对预训练进行分解？低秩预训练可以显著减少模型的参数数量，显著减少模型的预训练时间和成本。

在整个2010年代，许多人尝试训练低秩神经网络，这在研究如“深度神经网络训练的低秩矩阵分解与高维输出目标”(Sainath等人，2013)、“深度神经网络的半正交低秩矩阵分解”(Povey等人，2018)和“使用低秩扩展加速卷积神经网络”(Jaderberg等人，2014)中得到体现。

低秩分解在较小规模上已被证明是有效的。例如，通过应用各种分解策略，包括将 3×3 卷积替换为 1×1 卷积，SqueezeNet(Iandola等人，2016)在ImageNet上达到了AlexNet级别的准确率，同时使用的参数比AlexNet少50倍。

最近尝试训练低秩LLM的例子包括ReLoRA(Lialin等人，2023)和GaLore(Zhao等人，2024)。ReLoRA适用于参数最多13亿的基于transformer的模型。GaLore在10亿参数时达到了与全秩模型相当的性能，并在70亿参数时表现出有希望的性能。

可能在不太遥远的将来，研究人员将开发出一种方法，将低秩预训练扩展到数千亿参数。然而，如果Aghajanyan等人的论点是正确的——预训练隐式地压缩了模型的内在维度——全秩预训练仍然是必要的，以充分减少模型的内在维度，达到低秩分解可以工作的程度。研究究竟需要多少全秩训练才能切换到低秩训练将会很有趣。

LoRA配置

要应用LoRA，你需要决定将LoRA应用于哪些权重矩阵，以及每个分解的秩。本节将讨论每个决定的考虑因素。

LoRA可以应用于每个单独的权重矩阵。因此，LoRA的效率不仅取决于LoRA应用于哪些矩阵，还取决于模型的架构，因为不同的架构有不同的权重矩阵。

虽然已经有LoRA用于其他架构的例子，如卷积神经网络(Dutt等人, 2023; Zhong等人, 2024; Aleem等人, 2024)，但LoRA主要用于transformer模型。LoRA最常应用于注意力模块中的四个权重矩阵：查询(W_q)、键(W_k)、值(W_v)和输出投影(W_o)矩阵。

通常，LoRA均匀应用于模型中所有相同类型的矩阵。例如，将LoRA应用于查询矩阵意味着将LoRA应用于模型中的所有查询矩阵。

简单地说，你可以将LoRA应用于所有这些注意力矩阵。然而，通常你受到硬件内存的限制，只能容纳固定数量的可训练参数。给定固定的可训练参数预算，你应该将LoRA应用于哪些矩阵以最大化性能？

在微调GPT-3 175B时，Hu等人(2021)将可训练参数预算设定为18M，这是模型总参数数量的0.01%。这个预算允许他们应用LoRA到以下情况：

- 一个矩阵，秩为8
- 两个矩阵，秩为4
- 全部四个矩阵，秩为2

注意

GPT-3 175B有96个transformer层，模型维度为12,288。将秩=2的LoRA应用于所有四个矩阵将产生每层 $(12,288 \times 2 \times 2) \times 4 = 196,608$ 个可训练参数，或整个模型18,874,368个可训练参数。

他们发现，将秩=2的LoRA应用于所有四个矩阵在WikiSQL(Zhong等人, 2017)和MultiNLI(多类型自然语言推理)基准测试(Williams等人, 2017)上产生了最佳性能。表7-5显示了他们的结果。然而，作者建议，如果你只能选择两个注意力矩阵，查询和值矩阵通常会产生最佳结果。

表7-5. 可训练参数预算为18M时的LoRA性能。结果来自LoRA(Hu等人, 2021)。

Table 7-5. LoRA performance with the budget of 18M trainable parameters. Results from LoRA (Hu et al., 2021).

Number of trainable parameters = 18M							
Weight type	W_q	W_k	W_v	W_o	W_q, W_k	W_q, W_v	W_q, W_k, W_v, W_o
Rank r	8	8	8	8	4	4	2
WikiSQL ($\pm$ 0.5%)	70.4	70.0	73.0	73.2	71.4	73.7	73.7
MultiNLI ($\pm$ 0.1%)	91.0	90.8	91.0	91.3	91.3	91.3	91.7

经验观察表明，将LoRA应用于更多权重矩阵，包括前馈矩阵，会产生更好的结果。例如，Databricks表明，他们获得的最大性能提升来自将LoRA应用于所有前馈层 (Sooriyarachchi, 2023)。Fomenko等人(2024)注意到，基于前馈的LoRA可以补充基于注意力的LoRA，尽管在内存限制内，基于注意力的LoRA通常提供更大的效能。

LoRA的美妙之处在于，虽然其性能取决于其秩，但研究表明，对于许多用例来说，一个小的 r 值（例如4到64之间）通常就足够了。较小的 r 意味着较少的LoRA参数，这转化为较低的内存占用。

LoRA的作者观察到，让他们惊讶的是，增加 r 的值并不会增加微调性能。这一观察与Databricks的报告一致，即“将 r 增加到超过某个值可能不会产生任何明显的模型输出质量提升”(Sooriyarachchi, 2023)。有些人认为，更高的 r 甚至可能会产生负面影响，因为它可能导致过拟合。然而，在某些情况下，可能需要更高的秩。Raschka(2023)发现 $r = 256$ 在他的任务上取得了最佳性能。

你可以配置的另一个LoRA超参数是 α 值，它决定了在合并期间产品 WAB 应该对新矩阵贡献多少： $W' = W + \alpha \cdot WAB$ 。在实践中，我经常看到 α 被选择，使得 α/r 的比率通常在1:8和8:1之间，但最佳比率各不相同。例如，如果 r 很小，你可能希望 α 更大，如果 r 很大，你可能希望 α 更小。需要实验来确定最适合你的用例的 α/r 组合。

提供LoRA适配器服务

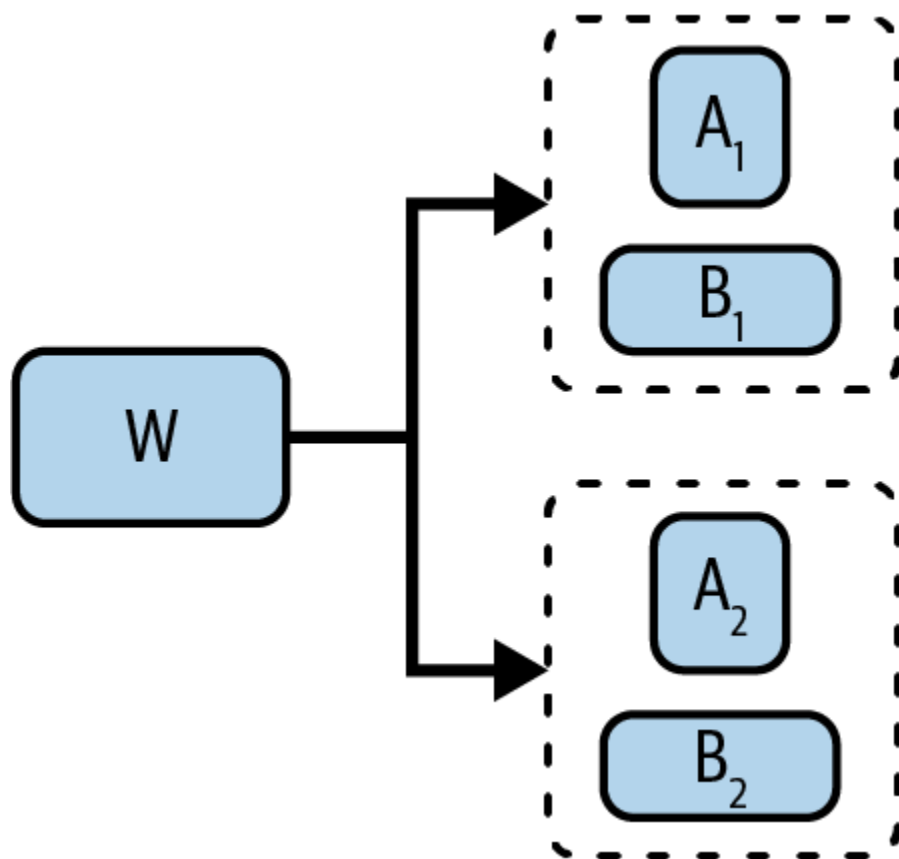
LoRA不仅让你可以使用更少的内存和数据微调模型，而且由于其模块化，它还简化了提供多个模型的服务。要理解这一好处，让我们研究如何为LoRA微调模型提供服务。

一般来说，有两种方式为LoRA微调模型提供服务：

1. 在提供微调模型服务之前，将LoRA权重A和B合并到原始模型中，创建新矩阵 W' 。由于在推理期间不执行额外的计算，因此不会增加额外的延迟。

2. 在提供服务期间保持W、A和B分离。将A和B合并回W的过程在推理期间发生，这增加了额外的延迟。

如果你只有一个LoRA模型需要提供服务，第一个选项通常更好，而第二个选项通常更适合多LoRA服务——为共享同一基础模型的多个LoRA模型提供服务。图7-12展示了如果你保持LoRA适配器分离，多LoRA服务的可视化。



对于多LoRA服务，虽然选项2增加了延迟开销，但它显著减少了所需的存储。考虑这样一个场景，你使用LoRA为每个客户微调一个模型。有100个客户，你最终得到100个微调模型，所有这些模型共享相同的基础模型。使用选项1，你必须存储100个全秩矩阵W'。使用选项2，你只需要存储一个全秩矩阵W，和100组较小的矩阵(A, B)。

为了说明这一点，假设原始矩阵W的维度是 4096×4096 (1680万参数)。如果LoRA的秩是8，A和B中的参数数量是 $4096 \times 8 \times 2 = 65,536$ ：

- 在选项1中，100个全秩矩阵W'总计 $1680万 \times 100 = 16.8$ 亿参数。
- 在选项2中，一个全秩矩阵W和100组小矩阵(A, B)总计： $1680万 + 65,536 \times 100 = 2330$ 万参数。

选项2还使得在任务之间切换更快。假设你目前使用这个客户的模型为客户X提供服务。要切换到为客户Y提供服务，而不是加载这个客户的完整权重矩阵，你只需要加载Y的LoRA适配器，这可以显著减少加载时间。虽然保持A和B分离会产生额外的延迟，但有优化技术可以最小化添加的延迟。本书的GitHub存储库包含如何做到这一点的演练。

多LoRA服务使得组合多个专门模型变得容易。与其为多个任务使用一个大而强大的模型，你可以为每个任务使用一个LoRA适配器。例如，苹果使用多个LoRA适配器来使同一个30亿参数的基础模型适应不同的iPhone功能(2024)。他们利用量化技术进一步减少这个基础模型和适配器的内存占用，允许在设备上提供所有这些服务。

LoRA适配器的模块化意味着LoRA适配器可以共享和重用。有公开可用的微调LoRA适配器，你可以像使用预训练模型一样使用它们。你可以在Hugging Face上找到它们，或者像AdapterHub这样的倡议。

你可能想知道：“LoRA听起来很棒，但有什么缺点？”LoRA的主要缺点是它不提供与完全微调一样强的性能。它也比完全微调更具挑战性，因为它涉及修改模型的实现，这需要理解模型的架构和编码技能。然而，这通常只是对不太流行的基础模型的问题。PEFT框架——如Hugging Face的PEFT、Axolotl、unsloth和LitGPT——很可能直接支持流行基础模型的LoRA。

量化的LoRA

LoRA的迅速兴起导致了众多LoRA变体的开发。有些旨在进一步减少可训练参数的数量。然而，如表7-6所示，LoRA适配器的内存与模型权重的内存相比微不足道。减少LoRA参数的数量只会减少整体内存占用的一小部分。

表7-6. LoRA权重所需的内存与模型权重所需的内存相比。

模型权重内存(16位)	LoRA可训练参数(r=2, 查询和键矩阵)	LoRA适配器内存(16位)
Llama 2(13B)	26 GB	3.28M
GPT-3(175B)	350 GB	18.87M

与其尝试减少LoRA的参数数量，你可以通过在微调期间量化模型的权重、激活和/或梯度来更有效地减少内存使用。LoRA的一个早期有希望的量化版本是QLoRA(Dettmers等人，2023)。在原始LoRA论文中，在微调期间，模型的权重使用16位存储。QLoRA将模型的权重存储在4位中，但

在在微调期间，QLoRA将模型的权重存储在4位中，但在计算前向和后向传递时将它们反量化(转换)回BF16。

QLoRA使用的4位格式是NF4(NormalFloat-4)，它基于预训练权重通常遵循中值为零的正态分布这一洞察来量化值。除了4位量化外，QLoRA还使用分页优化器，当GPU内存不足时，特别是对于长序列长度，自动在CPU和GPU之间传输数据。这些技术允许在单个48GB GPU上微调650亿参数的模型。

作者微调了各种模型，包括Llama 7B到65B，在4位模式下。由此产生的模型系列，称为Guanaco，在公共基准测试和比较评估上都显示了竞争性能。表7-7显示了2023年5月由GPT-4评判的Guanaco模型、GPT-4和ChatGPT的Elo评级。虽然Guanaco 65B没有超过GPT-4，但它通常被优先选择而不是ChatGPT。

表7-7. 使用GPT-4作为评判，2023年5月Guanaco模型与流行模型的Elo评级比较。实验来自QLoRA(Dettmers等人，2023)。

模型	大小	Elo
GPT-4	-	1348 ± 1
Guanaco 65B	41 GB	1022 ± 1
Guanaco 33B	21 GB	992 ± 1
Vicuna 13B	26 GB	974 ± 1
ChatGPT	-	966 ± 1
Guanaco 13B	10 GB	916 ± 1

Bard	-	902 ± 1
Guanaco 7B	6 GB	879 ± 1

QLoRA的主要限制是NF4量化成本高。虽然QLoRA可以减少内存占用，但由于量化和反量化步骤所需的额外时间，它可能会增加训练时间。

由于其节省内存的前景，量化LoRA是一个活跃的研究领域。除了QLoRA，量化LoRA工作还包括QA-LoRA(Xu等人，2023)、ModuLoRA(Yin等人，2023)和IR-QLoRA(Qin等人，2024)。

模型合并和多任务微调

如果微调允许你通过改变单个模型来创建自定义模型，模型合并允许你通过组合多个模型来创建自定义模型。模型合并提供了比单独微调更大的灵活性。你可以采用两个可用模型并将它们合并在一起，创建一个新的、希望更有用的模型。你也可以在将它们合并之前微调任何或所有组成模型。

虽然你不必进一步微调合并后的模型，但通过微调通常可以提高其性能。如果不进行微调，模型合并可以在没有GPU的情况下完成，这使合并对没有大量计算资源的独立模型开发者特别有吸引力。

模型合并的目标是创建一个单一模型，提供比单独使用所有组成模型更多的价值。增加的价值可能来自改进的性能。例如，如果你有两个在同一任务上擅长不同事情的模型，你可以将它们合并成一个在该任务上比两者都更好的单一模型。想象一个模型可以回答前60%的问题，另一个模型可以回答后60%的问题。组合在一起，也许它们可以回答80%的问题。

增加的价值也可能来自减少的内存占用，从而降低成本。例如，如果你有两个可以做不同任务的模型，它们可以合并成一个可以做两种任务但参数更少的模型。这对于基于适配器的模型特别有吸引力。给定在同一基础模型之上微调的两个模型，你可以将它们的适配器合并成一个单一适配器。

模型合并的一个重要用例是多任务微调。如果没有模型合并，想要为多个任务微调一个模型，你通常必须遵循以下方法之一：

同时微调 你创建一个包含所有任务示例的数据集，并在此数据集上微调模型，使模型同时学习所有任务。然而，因为同时学习多种技能通常更难，这种方法通常需要更多的数据和更多的训练。

顺序微调 你可以在每个任务上分别但顺序地微调模型。在任务A上训练模型后，再在任务B上训练它，依此类推。假设模型一次学习一个任务更容易。不幸的是，神经网络容易发生灾难性遗忘（

Kirkpatrick等人, 2016)。当模型在新任务上训练时, 它可能会忘记如何执行旧任务, 导致在早期任务上的性能显著下降。

模型合并提供了另一种多任务微调方法。你可以在不同任务上分别但并行地微调模型。完成后, 这些不同的模型被合并在一起。在每个任务上单独微调允许模型更好地学习该任务。因为没有顺序学习, 灾难性遗忘的风险较小。

当你必须将模型部署到设备上(如手机、笔记本电脑、汽车、智能手表和仓库机器人)时, 模型合并也很有吸引力。设备上部署通常具有挑战性, 因为设备内存容量有限。与其将多个用于不同任务的模型挤到一个设备上, 不如将这些模型合并成一个可以执行多个任务的模型, 同时需要更少的内存。

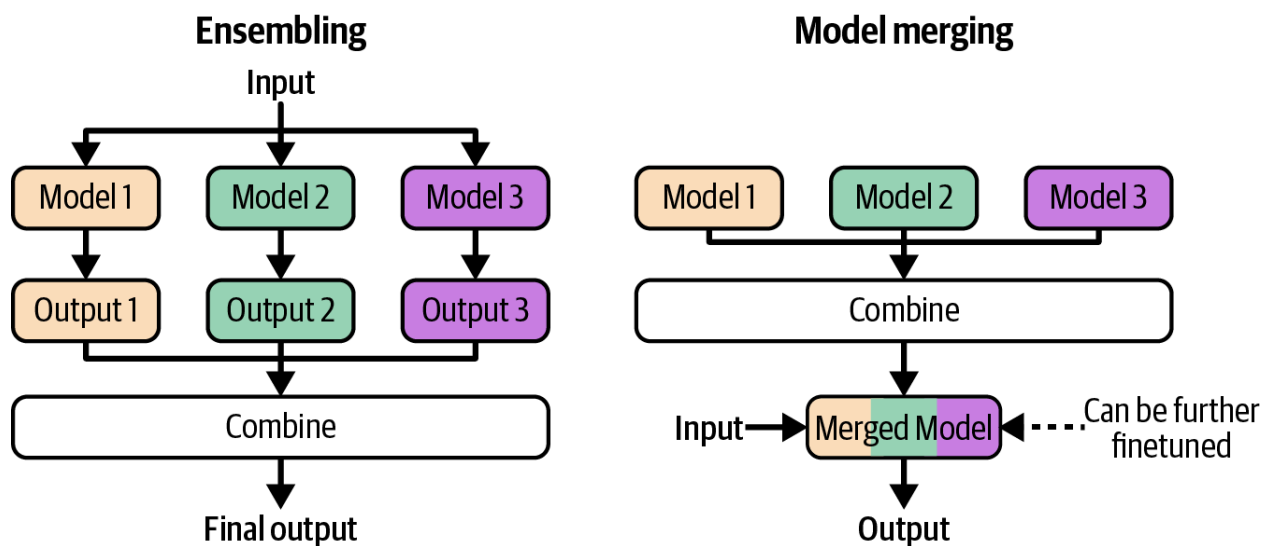
在以下情况下, 设备上部署是必要的: 数据不能离开设备(通常是出于隐私原因), 或者互联网访问有限或不可靠。设备上部署还可以显著降低推理成本。你能卸载到用户设备的计算越多, 你需要支付给数据中心的费用就越少。

模型合并是进行联邦学习(McMahan等人, 2016)的一种方式, 其中多个设备使用单独的数据训练同一模型。例如, 如果你将模型X部署到多个设备, X的每个副本可以从设备上的数据中单独继续学习。一段时间后, 你会有多个X的副本, 都在不同的数据上训练。你可以将这些副本合并成一个新的基础模型, 包含所有组成模型的学习。

将模型组合在一起以获得更好性能的想法始于模型集成方法。根据维基百科, 集成组合"多种学习算法, 以获得比单独使用任何组成学习算法都能获得的更好的预测性能"。如果模型合并通常涉及混合组成模型的参数, 集成通常只组合模型输出, 同时保持每个组成模型完整。

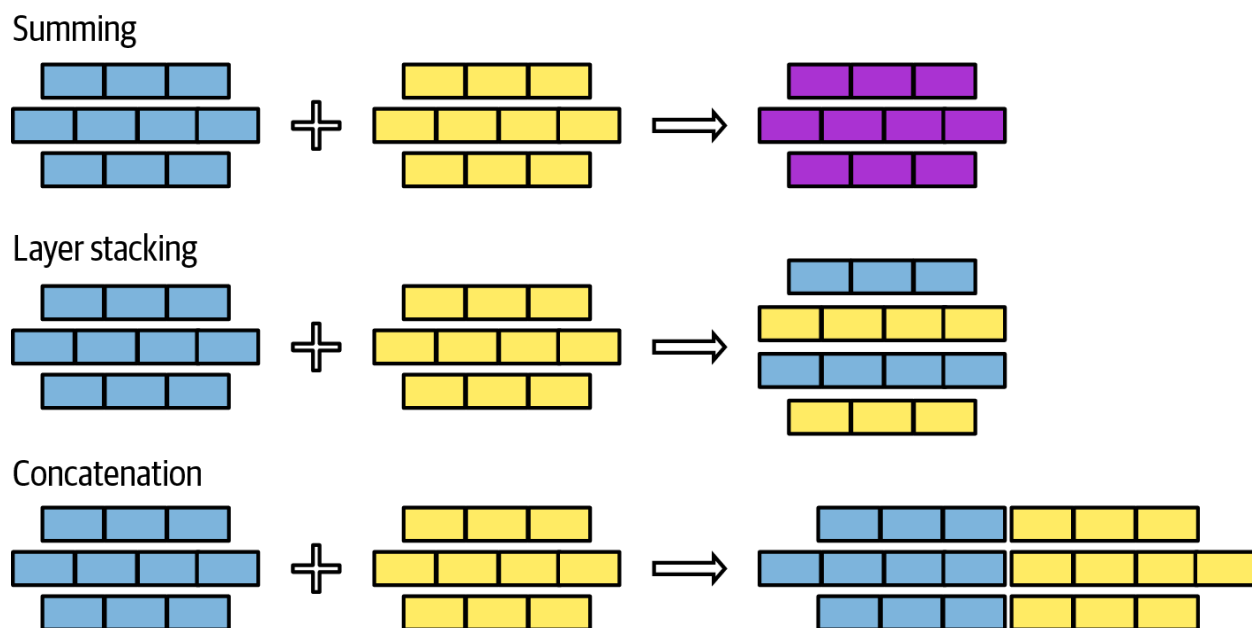
例如, 在集成中, 给定一个查询, 你可能使用三个模型生成三个不同的答案。然后, 基于这三个答案生成最终答案, 使用简单的多数投票或另一个可训练的ML模块。虽然集成通常可以提高性能, 但它的推理成本更高, 因为每个请求需要多次推理调用。

图7-13比较了集成和模型合并。就像模型集成曾经主导排行榜一样, Hugging Face的Open LLM排行榜上许多顶级模型都是合并模型。



许多模型合并技术都是实验性的，随着社区对底层理论有更好的理解，可能会过时。因此，我将重点关注高级合并方法，而不是任何单独的技术。

模型合并方法在如何组合组成参数上有所不同。这里介绍的三种方法是求和、层堆叠和连接。图 7-14 显示了它们的高级差异。



你可以在合并模型时混合这些方法，例如，对一些层求和，堆叠其他层。让我们探讨每种方法。

求和

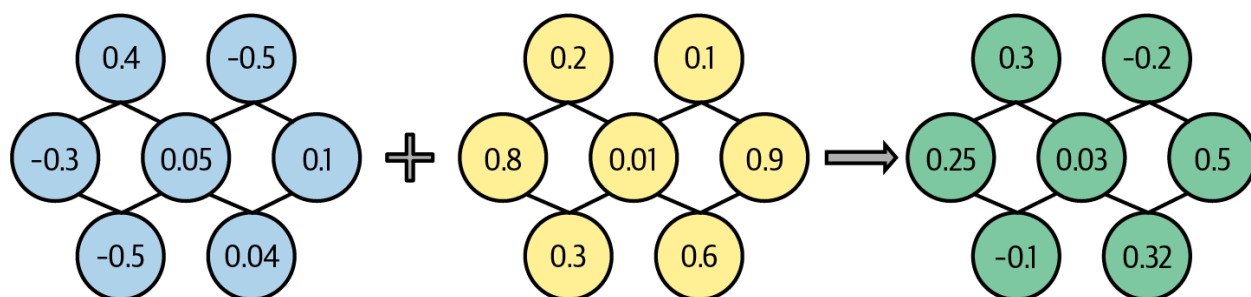
这种方法涉及将组成模型的权重值相加。我将讨论两种求和方法：线性组合和球面线性插值。如果两个模型中的参数在不同的尺度上，例如，一个模型的参数值比另一个大得多，你可以在求和之前重新缩放模型，使它们的参数值在相同的范围内。

线性组合

线性组合包括平均和加权平均。给定两个模型A和B，它们的加权平均是：

$$W' = (w_A \cdot W_A + w_B \cdot W_B) / (w_A + w_B)$$

图7-15显示了当 $w_A = w_B = 1$ 时如何线性组合两层。



考虑到其简单性，线性组合效果出奇地好。多个模型可以线性组合以创建更好的模型的想法早在20世纪90年代初就被研究过(Perrone, 1993)。线性组合经常用于联邦学习(Wang等人, 2020)。

你可以线性组合整个模型或模型的部分。模型汤(Wortsman等人, 2022)展示了如何通过平均多个微调模型的整个权重来提高准确性，而不增加推理时间。然而，更常见的是通过线性组合特定组件(如它们的适配器)来合并模型。

虽然你可以线性组合任何一组模型，但线性组合对于在同一基础模型之上微调的模型最有效。在这种情况下，线性组合可以通过任务向量的概念来理解。其想法是，一旦你为特定任务微调了模型，从中减去基础模型应该给你一个捕捉任务本质的向量。任务向量也称为增量参数。如果你使用LoRA进行微调，你可以从LoRA权重构建任务向量。

任务向量允许我们进行任务算术(Ilharco等人, 2022)，例如加两个任务向量来组合任务能力，或减去一个任务向量来减少特定能力。任务减法对于移除不良模型行为很有用，例如在预训练期间获得的侵入性能力(如面部识别)或偏见。

当要合并的组件具有相同的架构和相同的大小时，线性组合很简单。然而，它也适用于不共享相同架构或相同大小的模型。例如，如果一个模型的层比另一个模型的层大，你可以将一个或两层投影到相同的维度。

一些人提出在平均之前对模型进行对齐，以确保功能相关的参数一起平均，例如在"通过最优传输进行模型融合"(Singh和Jaggi, 2020)、"Git Re-Basin: 在排列对称性模下合并模型"(Ainsworth等

人, 2022)和"通过匹配任务参数子空间中的模型进行合并"(Tam等人, 2023)中。虽然组合对齐的参数是有意义的, 但对齐参数可能很难做到, 因此这种方法不如天真的线性组合常见。

球面线性插值(SLERP)

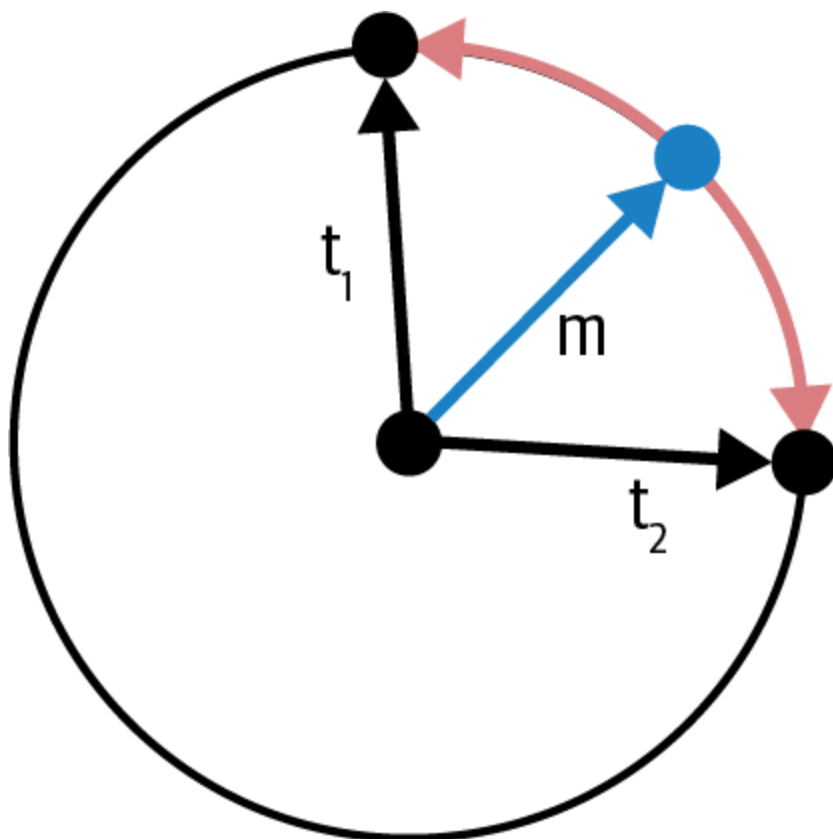
另一种常见的模型求和方法是SLERP, 它基于同名的数学运算符, 球面线性插值。

注意

插值意味着根据已知值估计未知值。在模型合并的情况下, 未知值是合并后的模型, 已知值是组成模型。线性组合是一种插值技术。SLERP是另一种。

因为SLERP的公式很数学, 而且模型合并工具通常会为你实现它, 所以我不会在这里详细介绍。直观地说, 你可以把每个要合并的组件(向量)看作是球体上的一个点。要合并两个向量, 你首先沿着球体表面画出这两点之间的最短路径。这类似于沿着地球表面画出两个城市之间的最短路径。这两个向量的合并向量是沿着它们最短路径上的一个点。该点确切地落在路径上的位置取决于插值因子, 你可以设置在0到1之间。小于0.5的因子值使合并向量更接近第一个向量, 这意味着第一个任务向量将对结果贡献更多。0.5的因子意味着你选择一个正好在中间的点。这个中间点是图7-16中的蓝色点。

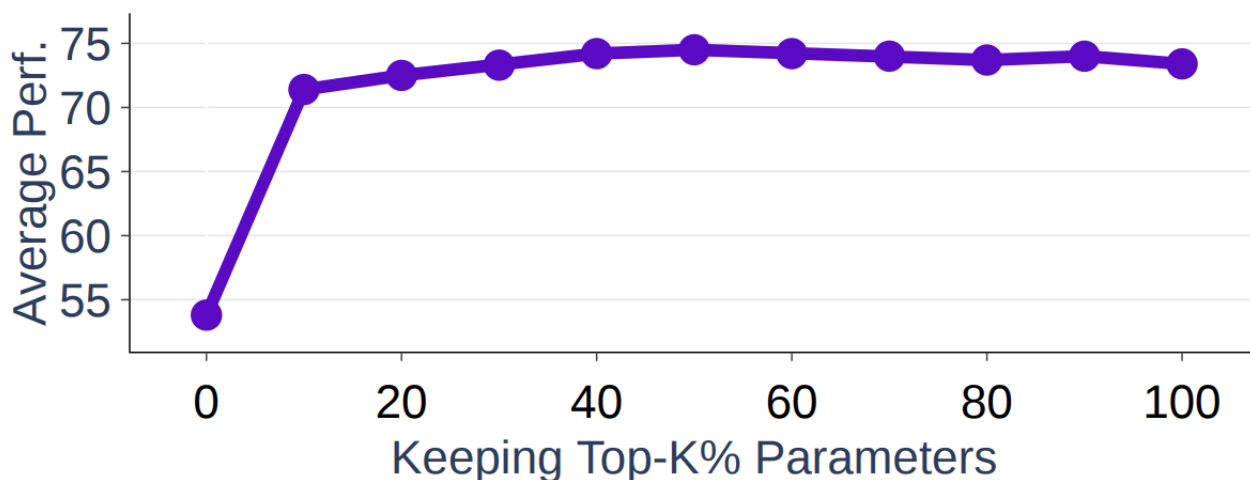
SLERP作为一种数学运算, 只定义了两个向量, 这意味着你一次只能合并两个向量。如果你想合并两个以上的向量, 你可以按顺序进行SLERP, 即将A与B合并, 然后将结果与C合并。



修剪冗余的任务特定参数

在微调期间，许多模型参数都被调整。然而，这些调整中的大多数是微不足道的，对模型在任务上的性能没有显著贡献。不对模型性能有贡献的调整被视为冗余。

在论文“TIES-合并：解决合并模型时的干扰”中，Yadav等人(2023)表明，你可以重置大部分任务向量参数，性能几乎不会下降，如图7-17所示。重置意味着将微调参数更改为基础模型中的原始值，有效地将相应的任务向量参数设置为零。（回想一下，任务向量可以通过从微调模型中减去基础模型来获得。）



这些冗余参数，虽然对一个模型无害，但可能对合并后的模型有害。诸如TIES(Yadav等人, 2023)和DARE(Yu等人, 2023)等合并技术在合并任务向量之前先从任务向量中修剪冗余参数。两篇论文都表明，这种做法可以显著提高最终合并模型的质量。要合并的模型越多，修剪就越重要，因为一个任务中的冗余参数有更多机会干扰其他任务。

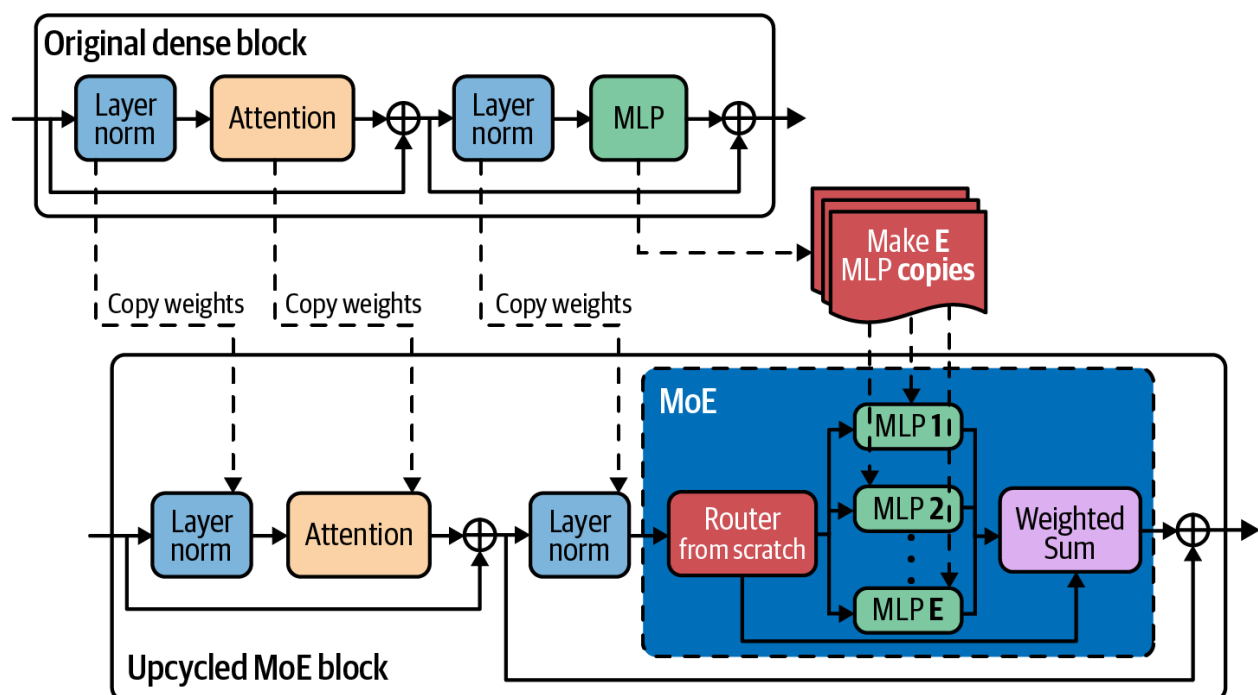
层堆叠

在这种方法中，你从一个或多个模型中取不同的层并将它们堆叠在一起。例如，你可以从模型1取第一层，从模型2取第二层。这种方法也称为通过或弗兰肯合并。它可以创建具有独特架构和参数数量的模型。与通过求和合并的方法不同，通过层堆叠产生的合并模型通常需要进一步微调才能获得良好的性能。

弗兰肯合并的一个早期成功是Goliath-120B(alpindale, 2023)，它由两个微调的Llama 2-70B模型Xwin和Euryale合并而成。它从每个模型中取了72个(共80个)层并将它们合并在一起。

层堆叠可以用来训练专家混合(MoE)模型，如“稀疏回收：从密集检查点训练专家混合”中所介绍的(Komatsuzaki等人, 2022)。与其从头开始训练MoE，不如采用预训练模型并复制某些层或模块。然后添加一个路由器，将每个输入发送到最合适的副本。然后，你进一步训练合并后的模型以及路由器，以完善它们的性能。图7-18说明了这个过程。

Komatsuzaki等人表明，层堆叠可以产生性能优于从头开始训练的MoE模型。使用这种方法，Together AI将六个较弱的开源模型混合在一起，创建了Mixture-of-Agents，在某些基准测试中达到了与OpenAI的GPT-4o相当的性能(Wang等人, 2024)。

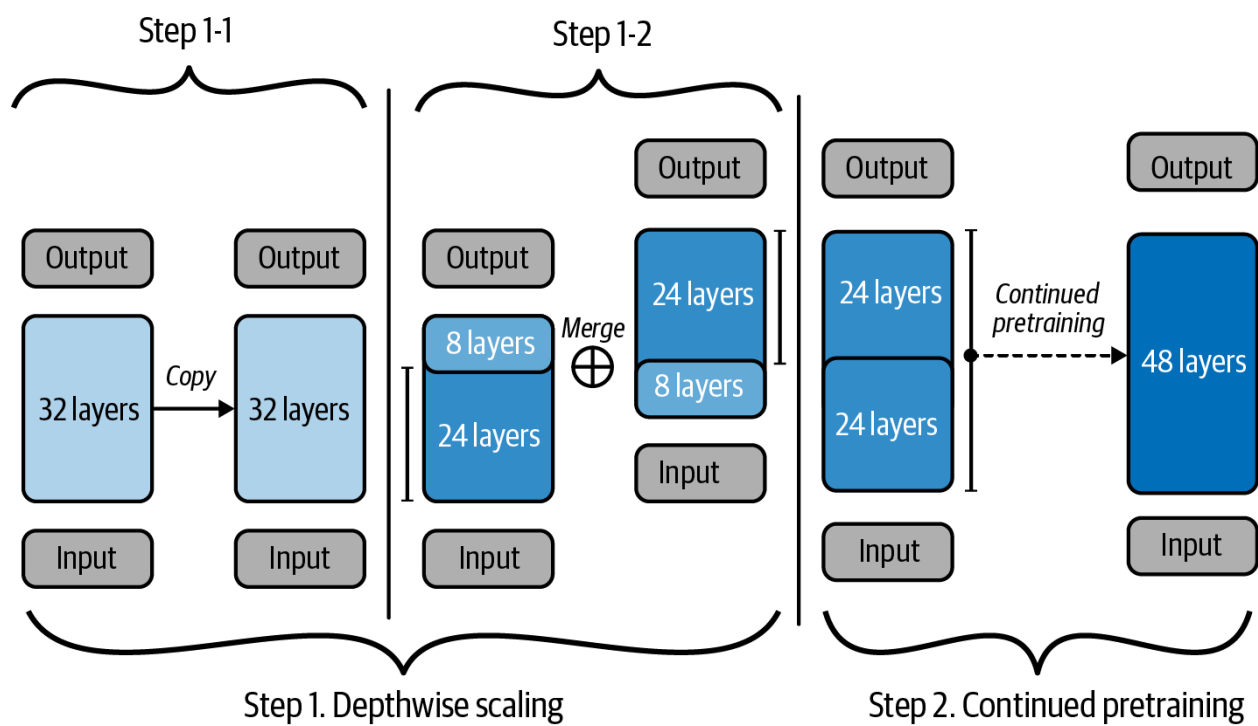


层堆叠的一个有趣用例是模型上扩。模型上扩是关于如何使用更少资源创建更大模型的研究。有时，你可能想要一个比现有模型更大的模型，可能是因为更大的模型提供更好的性能。例如，你的团队可能最初训练了一个适合40GB GPU的模型。然而，你获得了一台新机器，有80GB内存，允许你提供更大的模型服务。与其从头开始训练新模型，不如使用层堆叠从现有模型创建更大的模型。

模型上扩的一种方法是深度扩展。Kim等人(2023)使用这种技术从一个有32层的70亿参数模型创建了SOLAR 10.7B。程序如下：

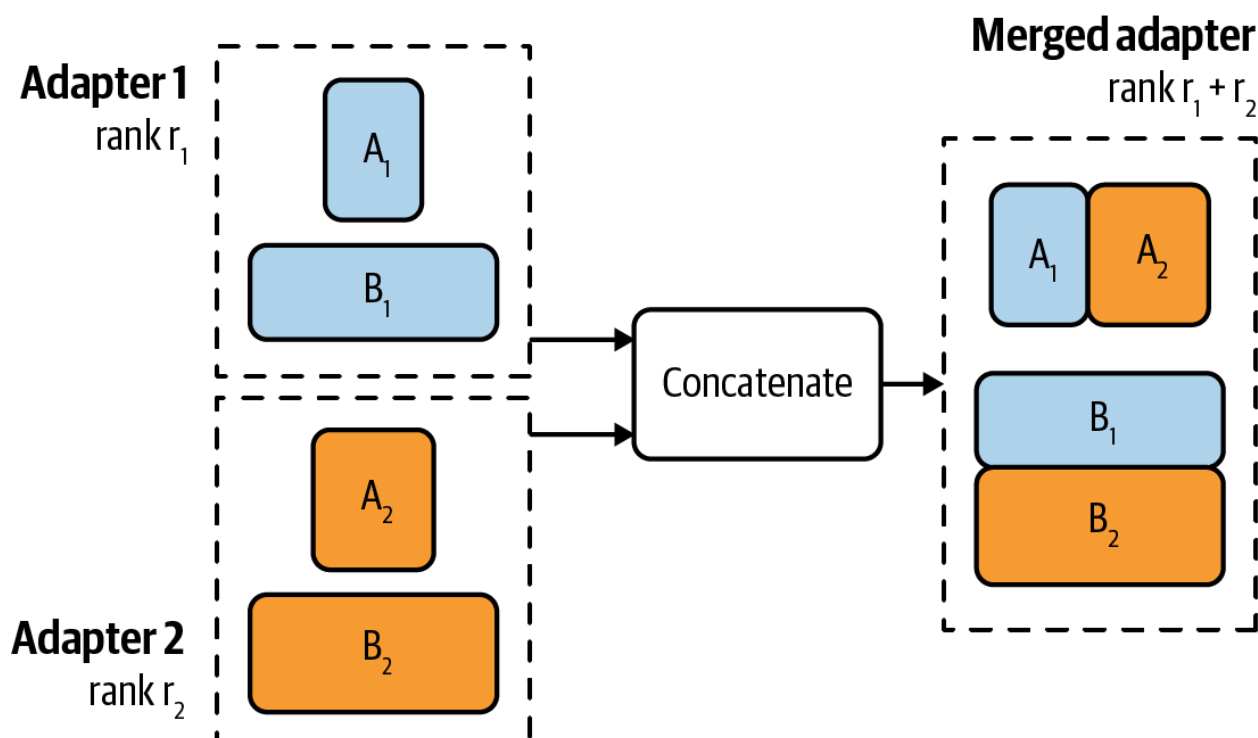
1. 复制原始预训练模型。
2. 通过求和某些层(将两层求和变为一层)并堆叠其余层来合并这两个副本。要求和的层经过精心选择，以匹配目标模型大小。对于SOLAR 10.7B，16层被求和，最终模型有 $32 \times 2 - 16 = 48$ 层。
3. 进一步训练这个上扩的模型，以达到目标性能。

图7-19说明了这个过程。



连接

除了以不同方式将组成模型的参数相加之外，你也可以连接它们。合并组件的参数数量将是所有组成组件参数数量的总和。如果你合并两个秩为 r_1 和 r_2 的LoRA适配器，合并适配器的秩将是 $r_1 + r_2$ ，如图7-20所示。



不推荐连接，因为与单独提供不同模型服务相比，它不会减少内存占用。连接可能会提供更好的性能，但增量性能可能不值得额外参数的数量。

微调策略

本章讨论了多种微调方法，它们解决什么问题，以及它们如何工作。在最后一节中，我将重点关注更实用的微调策略。

微调框架和基础模型

虽然微调周围的许多事情——决定是否微调、获取数据和维护微调模型——都很困难，但实际的微调过程更直接。有三件事你需要选择：基础模型、微调方法和微调框架。

基础模型

第4章已经介绍了适用于基于提示的方法和微调的模型选择标准。一些讨论的标准包括模型大小、许可证和基准性能。在AI项目开始时，当你仍在探索任务的可行性时，从你能负担得起的最强大的模型开始是有用的。如果这个模型难以产生好的结果，更弱的模型可能表现更差。如果最强的模型满足你的需求，你可以探索更弱的模型，使用初始模型作为比较基准。

对于微调，不同项目的起始模型各不相同。OpenAI的微调最佳实践文档给出了两条开发路径的例子：进阶路径和蒸馏路径。

进阶路径看起来像这样：

1. 使用最便宜、最快的模型测试你的微调代码，确保代码按预期工作。
2. 通过微调一个中等模型来测试你的数据。如果训练损失随着更多数据而不下降，可能有问题。
3. 用最好的模型运行更多实验，看看你能推动性能多远。
4. 一旦有了好的结果，对所有模型进行训练运行，映射出价格/性能前沿，并选择对你的用例最有意义的模型。

蒸馏路径可能如下：

1. 从小数据集和你能负担得起的最强模型开始。用这个小数据集训练可能的最佳模型。因为基础模型已经很强，它需要更少的数据就能达到良好的性能。
2. 使用这个微调模型生成更多的训练数据。
3. 使用这个新数据集来训练一个更便宜的模型。

因为微调通常在提示工程实验之后进行，理想情况下，到开始微调时，你应该对不同模型的行为有相当好的理解。你应该根据这种理解规划微调开发路径。

微调方法

回想一下，像LoRA这样的适配器技术成本效益高，但通常不能提供与完全微调相同水平的性能。如果你刚开始微调，尝试像LoRA这样的东西，以后再尝试完全微调。

使用的微调方法也取决于你的数据量。根据基础模型和任务的不同，完全微调通常需要至少数千个例子，通常更多。然而，PEFT方法可以在更小的数据集上表现良好。如果你的数据集很小，例如几百个例子，完全微调可能不会比LoRA表现更好。

在决定微调方法时，考虑你需要多少微调模型以及你想如何提供服务。基于适配器的方法（如LoRA）允许你更有效地提供共享同一基础模型的多个模型服务。使用LoRA，你只需要提供单个完整模型服务，而完全微调需要提供多个完整模型服务。

微调框架

微调的最简单方法是使用微调API，你可以上传数据，选择基础模型，并获得微调后的模型。像模型推理API一样，微调API可以由模型提供商、云服务提供商和第三方提供商提供。这种方法的限制是你受限于API支持的基础模型。另一个限制是API可能不会暴露所有可用于最佳微调性能的

调整旋钮。微调API适合那些想要快速简单的东西的人，但对那些想要更多定制的人可能会令人沮丧。

你也可以使用许多可用的优秀微调框架之一进行微调，如LLaMA-Factory、unsloth、PEFT、Axoloti和LitGPT。它们支持广泛的微调方法，特别是基于适配器的技术。如果你想进行完全微调，许多基础模型在GitHub上提供开源训练代码，你可以克隆并用自己的数据运行。Llama Police有一个更全面和最新的微调框架和模型存储库列表。

自己进行微调给了你更多的灵活性，但你需要提供必要的计算资源。如果你只使用基于适配器的技术，中级GPU可能足够用于大多数模型。如果你需要更多计算能力，你可以选择与你的云提供商无缝集成的框架。

要使用多台机器微调模型，你需要一个帮助你进行分布式训练的框架，如DeepSpeed、PyTorch Distributed和ColossalAI。

微调超参数

根据基础模型和微调方法，有许多超参数可以调整以提高微调效率。对于你用例的具体超参数，请查看你使用的基础模型或微调框架的文档。在这里，我将介绍一些经常出现的重要超参数。

学习率

学习率决定了模型参数在每个学习步骤中应该变化多快。如果你认为学习是找到通往目标的路径，学习率就是步长。如果步长太小，可能需要太长时间才能到达目标。如果步长太大，你可能会越过目标，因此模型可能永远不会收敛。

不存在通用的最佳学习率。你需要尝试不同的学习率，通常在 $1e-7$ 到 $1e-3$ 的范围内，看看哪一个效果最好。一种常见的做法是取预训练阶段结束时的学习率，并将其乘以0.1到1之间的常数。

损失曲线可以给出关于学习率的提示。如果损失曲线波动很大，学习率可能太大。如果损失曲线稳定但很长时间才减少，学习率可能太小。尽可能将学习率增加到损失曲线保持稳定的程度。

你可以在训练过程中改变学习率。你可以在开始时使用较大的学习率，在结束时使用较小的学习率。决定学习率应该如何在整个训练过程中变化的算法称为学习率调度。

批处理大小

批处理大小决定了模型在每个步骤中从多少个例子学习以更新其权重。太小的批处理大小，如少于八个，可能导致不稳定的训练。较大的批处理大小有助于从不同例子中聚合信号，导致更稳定和可靠的更新。

一般来说，批处理大小越大，模型通过训练例子的速度就越快。然而，批处理大小越大，运行模型所需的内存就越多。因此，批处理大小受到你使用的硬件限制。

这就是你看到成本与效率的权衡。更昂贵的计算允许更快的微调。

截至目前，计算仍然是微调的瓶颈。通常，模型如此之大，内存如此受限，只能使用小批处理大小。这可能导致模型权重更新不稳定。为解决模型权重更新不稳定的问题，你可以使用梯度累积技术。与其在每个批次后更新模型权重，你可以在多个批次中累积梯度，并在积累了足够可靠的梯度后再更新模型权重。这种技术称为梯度累积。

训练轮数

一个轮次(epoch)是对训练数据的一次完整遍历。训练轮数决定每个训练示例被训练的次数。

小数据集可能需要比大数据集更多的轮次。对于有数百万示例的数据集，1-2个轮次可能就足够了。有数千个示例的数据集在4-10个轮次后可能仍然会看到性能提升。

训练损失和验证损失之间的差异可以给出关于轮次的提示。如果训练损失和验证损失都在稳步下降，模型可以从更多轮次(和更多数据)中受益。如果训练损失仍在下降但验证损失增加，模型可能正在过拟合训练数据，你可能需要尝试降低轮次数。

提示损失权重

对于指令微调，每个示例由一个提示和一个响应组成，这两者在训练期间都可以对模型的损失有所贡献。然而，在推理期间，提示通常由用户提供，模型只需生成响应。因此，响应标记应该比提示标记对训练期间的模型损失贡献更多。

提示模型权重决定了提示应该对这种损失贡献多少，相对于响应。如果这个权重是100%，提示对损失的贡献与响应一样多，意味着模型从两者中平等学习。如果这个权重是0%，模型只从响应中学习。通常，默认情况下这个权重被设置为10%，意味着模型应该从提示中学习一些，但主要从响应中学习。

总结

除了评估章节之外，微调是最具挑战性的章节。它涉及广泛的概念，既有旧的(迁移学习)，也有新的(PEFT)，基础的(低秩分解)和实验性的(模型合并)，数学的(内存计算)和策略性的(超参数调整)。将所有这些不同方面组织成一个连贯的结构，同时保持它们的可访问性是困难的。

微调过程本身并不难。许多微调框架为你处理训练过程。这些框架甚至可以建议具有合理默认超参数的常见微调方法。

然而，围绕微调的背景是复杂的。它始于你是否应该微调模型。本章从微调的理由和不微调的理由开始。它还讨论了一个我被问过多次的问题：何时微调，何时做RAG。

在早期，微调类似于预训练——两者都涉及更新模型的全部权重。然而，随着模型规模的增加，完全微调对大多数从业者来说变得不切实际。在微调期间更新的参数越多，微调需要的内存就越多。大多数从业者无法获得足够的资源(硬件、时间和数据)来对基础模型进行完全微调。

许多微调技术都是出于同样的动机开发的：在最小内存占用上实现强大性能。例如，PEFT通过减少可训练参数的数量来减少微调的内存需求。另一方面，量化训练通过减少表示每个值所需的位数来缓解这种内存瓶颈。

在概述PEFT后，本章深入研究了LoRA——它为什么有效以及如何工作。LoRA有许多特性使其在从业者中受欢迎。除了参数高效和数据高效外，它还具有模块化特性，使得提供和组合多个LoRA模型变得更加容易。

组合微调模型的想法引出了模型合并；其目标是将多个模型合并为一个比这些模型单独使用更好的模型。本章讨论了模型合并的许多用例，从设备上部署到模型上扩，以及模型合并的一般方法。

我经常从从业者那里听到的评论是，微调很容易，但获取微调数据很难。获取高质量的标注数据，特别是指令数据，具有挑战性。下一章将深入探讨这些挑战。